Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Resumen Sistemas Operativos 2

2024

Índice

1. Introducción **3**
1.1. Definición y funciones . . . 3
1.2. Comienzo de los SO . . . 3
1.3. Multiprogramación y tiempo compartido . . . 4
1.4. Organización de los SO . . . 5

2. Relación con el hardware **5**
2.1. Jerarquía de almacenamiento . . . 6
2.2. Registros . . . 6
2.3. Acceso directo a memoria (DMA) . . . 6
2.4. Llamadas al sistema . . . 7

3. Administración de procesos **7**
3.1. Estados de un proceso . . . 7
3.2. Diferencias con hilos . . . 8
3.3. Comunicación entre procesos: Señales . . . 8

4. Planificación de procesos **9**
4.1. Procesos cortos vs largos . . . 10
4.2. Parámetros de respuesta . . . 10
4.3. Algoritmos de planificación . . . 11
4.4. Planificación de hilos . . . 16
4.5. Planificación de multiprocesadores . . . 16
4.6. Inversión de prioridades . . . 17

5. Linux Scheduler **17**
5.1. Tareas . . . 18
5.2. Prioridades . . . 18
5.3. Historia de los planificadores . . . 19
5.4. CPU ideal . . . 19

6. Administración de memoria **20**
6.1. Funciones y operaciones . . . 20

6.2. Asignación de memoria contigua 22
6.3. Segmentación 23
6.4. Paginación 23
6.5. Memoria compartida 25
6.6. Memoria virtual 26
6.7. Algoritmos de reemplazo de páginas 27
6.8. Asignación de marcos 31
6.9. Hiperpaginación, Sobrepraginación o Thrashing 32

7. Memoria virtual en Linux **32**
7.1. Organización de la memoria física 32
7.2. Ubicación de memoria y gestión de memoria libre 34
7.3. Reclamo de páginas - Out of Memory 34
7.4. Swap en Linux 36

8. Sistemas de archivos **36**
8.1. Conceptos básicos 36
8.2. Directorios 39
8.3. Discos duros 41
8.4. Algunos sistemas de archivos 42
8.5. Control de acceso 48
8.6. Sistemas de archivos remotos 49

9. Sistemas Distribuidos **51**
9.1. Algoritmos distribuidos 52
9.2. Sistema Operativo Distribuido 53
9.3. Detección de deadlocks 53

10. Bibliografía **54**

1. Introducción

1.1. Definición y funciones

Un sistema operativo es el programa principal que se ejecuta en una computadora. Es importante comprender su funcionamiento básico y las principales alternativas que ofrecen muchos de sus puntos, además de saber diseñar algoritmos y procesos que se ajusten mejor al SO en que vayan a ejecutarse. El núcleo es una parte importante del SO, que es la que se encarga de trabajar en modo privilegiado. Es más, la mayor parte de la funcionalidad del SO se observa en el núcleo, se realizan en él tareas que un programa de usuario no puede hacer.

El SO es el único programa que interactúa directamente con el hardware de la computadora, sus funciones principales son 3:

- **Abstracción**: Se encarga de proporcionar una serie de abstracciones para que los programadores puedan enfocarse en resolver las necesidades particulares de sus usuarios. Generar una máquina más usable, poder hablar de archivos y de directorios, no de mapas de bits. O usar la abstracción de sockets en vez de trabajar con algo a más bajo nivel. Todo lo que es crear procesos y ejecutarlos concurrentemente, al igual que usar alarmas o señales, es una abstracción del SO. Esto sirve para que todo sea más portable, porque no se usan en los programas de usuarios especificaciones de hardware por ejemplo. archivos, sockets, procesos.

- **Administración de recursos**: Al gestionar toda la asignación de recursos, el SO puede implementar políticas que los asignen de forma efectiva y acorde a las necesidades establecidas para dicho sistema. Cuando tengo varios programas que quiero ejecutar en un mismo momento, son procesos que se ejecutan en la misma memoria RAM, utilizan el mismo disco y el mismo procesador. El núcleo se encarga de separar estos recursos limitados entre los procesos y buscar el mejor rendimiento. Además, algunos recursos tienen acceso exclusivo, y debe controlar ese acceso.

- **Aislamiento**: Permite que cada proceso y cada usuario no tenga que preocuparse por otros que estén usando el mismo sistema, idealmente la experiencia del usuario será la misma que si el sistema estuviera exclusivamente dedicado a su atención. El proceso ve que tiene un CPU dedicado para él, y una memoria para cada proceso (virtual), además todo lo que tiene que ver con el swap. Que un programa no pueda acceder a la memoria de otro programa, ni que puedan acceder juntos al mismo dispositivo.

1.2. Comienzo de los SO

En una primer época, comenzó simplemente a automatizarse el procesamiento de los programas. Surgió el **procesamiento por lotes**, con el cual en lugar de ejecutar programas "de a uno", los usuarios preparaban un conjunto de trabajos o tareas (llamados "lotes") y los

cargaban en el sistema informático, en donde se los procesaba secuencialmente. El monitor de lotes cargaba cada trabajo en la memoria, lo ejecutaba y luego pasaba al siguiente trabajo en la cola. Este monitor fue el primer indicio de lo que hoy se conoce como un SO. Claramente se fue sofisticando al implementar mejoras como protecciones que evitaran corrupciones entre trabajos. Estos monitores, comenzaron a necesitar modificaciones en hardware para poder proveer características de seguridad, y es cuando comienza a aparecer esta gestión de recursos y abstracción del SO.

Tiempo después, se inventó el **spool**, un mecanismo de entrada/salida que permitía que una computadora leyera las tarjetas y las fuera convirtiendo a cinta magnética, la cuál era leída por la computadora principal.

Cuando comenzaron a aparecer los **sistemas multiprogramados**, en los cuales se permitía ejecutar varios procesos al mismo tiempo, se necesitó de un hardware más específico para poder permitir la protección de recursos. Esta protección se encontró en la **MMU** (Memory Management Unit).

1.3. Multiprogramación y tiempo compartido

Son dos modelos de gestión de recursos en sistemas informáticos. La multiprogramación se desarrolló cuando las computadoras se utilizaban principalmente para ejecutar un solo programa a la vez, y se encontraba mucho tiempo ocioso entre las operaciones de entrada/-salida. Surgió como una forma de utilizar más eficientemente los recursos de la computadora al permitir la ejecución simultánea de varios programas. Se cargaban varios programas en la memoria y se alternaba entre ellos de manera cooperativa. Su objetivo era priorizar el uso eficiente de la CPU y la memoria, permitiendo que múltiples programas compartan estos recursos de manera simultánea.

Por otro lado, el tiempo compartido surgió como una evolución de la multiprogramación a medida que las computadoras se volvieron más poderosas y accesibles, utilizando multitarea apropiativa. Permite que múltiples usuarios accedan a la computadora a través de terminales, y el SO divide el tiempo de procesador entre los distintos usuarios de manera que cada usuario tenga la impresión de tener la computadora para sí mismo.

- **Multitarea cooperativa**: cada proceso tiene control del CPU hasta que haga una llamada a sistema, ceda el paso o termine.
- **Multitarea apropiativa**: el reloj del sistema interrumpe periódicamente a los procesos y transfiere el control al SO para que elija un (a veces nuevo) proceso al cual darle el uso del CPU. Se define el concepto de `tick`: unidad de medida del tiempo, interrupción del reloj que manda señales al procesador, utilizado para sincronizar o temporizar.

Un cambio importante entre la multiprogramación y el tiempo compartido es que la velocidad del cambio entre una tarea y otra es mucho más rápida en el segundo, pues en un sistema

interactivo se busca dar la ilusión de uso exclusivo de la computadora. Sin embargo, se comenzó a considerar que diferentes tipos de procesos podían tener diferente importancia, por lo cual se les debería asignar prioridades a los mismos.

1.4. Organización de los SO

Como dijimos, podemos definir a un SO como un programa que ejecuta otros programas y les provee un conjunto de interfaces para que puedan aprovechar los recursos de cómputo. Es decir, tiene un núcleo, que se encarga de ejecutar los programas de usuario, en instancias llamadas procesos. De esta distinción surgen dos modos o espacios de ejecución:

- Modo núcleo: Lo que se ejecuta en este modo tiene acceso libre a todo el hardware. Es decir, tiene privilegios. Acá se ubica el núcleo.
- Modo usuario: Lo que se ejecuta en este modo necesita pedir a software del otro modo que actúe de intermediario. Es decir, tienen permisos restringidos. Acá se ubican los programas de usuario.

Hay dos formas primarias de organización interna del SO. Su distinción surge de cuanta funcionalidad se desea incluir en el núcleo, y cuantas tareas podrían moverse a espacio usuario. Mínimamente, se necesita que el kernel tenga alguna gestión de procesos, un manejo básico de espacios de direcciones y comunicación entre procesos. Luego, surgen las diferencias.

Por un lado tenemos los **sistemas monolíticos**, en los cuales hay un sólo proceso privilegiado (el SO) que opera en modo supervisor. Por otro, están los **sistemas microkernel**, en el cual el núcleo del SO se mantiene en el mínimo posible de funcionalidad, descargando en procesos especiales sin privilegios las tareas que implementan el acceso a dispositivos y las diversas políticas de uso del sistema.

La principal ventaja de un sistema monolítico, es la simplificación de una gran cantidad de mecanismos de comunicación, lo cual lleva también a una mayor velocidad de ejecución. En cambio, los sistemas microkernel tienen la ventaja de ser más limpios y elegantes, lo cual facilita la comprensión por separado de cada una de sus piezas. Esto permite que si una parte falla, el núcleo pueda reemplazarla o bien repararla. El sistema es más robusto, más fácil de extender, depurar y mantener. Estas formas de organización se estudiarán en detalle en la sección 10 del apunte.

2. Relación con el hardware

Todas las computadoras están compuestas por una unidad de procesamiento, y dispositivos que permiten ingresar datos o bien obtener resultados. ¿Qué es la unidad de procesamiento? Es la parte fundamental de todo sistema de cómputo, encargada de ejecutar tanto los programas de usuario como el SO en sí.

Las computadoras que siguen la arquitectura *Von Neumann* podrían resumir su operación general a alimentar la CPU con los datos e instrucciones almacenados en memoria. Sin embargo, esta clase de arquitectura que tiene la particularidad de utilizar la memoria tnato para el programa como para sus datos, conlleva un problema denominado *cuello de botella de Von Neumann*.

Este cuello de botella refiere a la diferencia entre la velocidad que puede adquirir la CPU a comparación de la velocidad de la memoria. Esto ocurre porque los procesadores son cada vez más rápidos mientras que la RAM aumenta su velocidad a un ritmo mucho menor.

2.1. Jerarquía de almacenamiento

Una respuesta parcial al problema del cuello de botella de *Von Neumann*, fue la implementación de jerarquías de almacenamiento. Gracias a ella, en un mismo sistema se encontraban desde áreas de memoria muy pequeñas y caras, pero extremadamente rápidas a porciones de mayor tamaño, pero menor velocidad y costo.

Por un lado se encuentran los registros, que son parte integral del procesador. Por otro, está la memoria principal, la cual es posible referenciar directamente desde el procesador (se dice que están a un paso de distancia). Por otro lado, se encuentra la caché del procesador, la cual no es utilizada de manera explícita por el CPU, si no manejada por los controladores de acceso a memoria. Finalmente, se encuentra el disco, pero el procesador no puede acceder a él de manera directa, si no que se realiza la instrucción transfiriéndose el contenido necesario a memoria.

2.2. Registros

Representan la memoria más rápida de la computadora. Están ubicados en cada uno de los núcleos de cada CPU, y el estado del CPU a cada momento está determinado por el contenido de los mismos.

En las arquitecturas modernas, existen registros de propósito general, regístros índices (por ejemplo el `stack pointer`), regístros de bandera, de segmentos (como el `data segment` o `code segment`), y un regístro apuntador de instrucciones (`instruction pointer`).

2.3. Acceso directo a memoria (DMA)

Cuando un dispositivo de entrada/salida (E/S) necesita transferir datos a la memoria principal (RAM), sin el uso de DMA, debe generar una interrupción cada vez que tiene un dato listo para ser transferido. La misma será recibida por el procesador, quien tendrá que detener su tarea actual, y ejecutar una interrupción para transferir el dato desde el dispositivo a la memoria. Este proceso de interrupciones frecuentes y manejo de cada transferencia individual por el procesador puede crear un cuello de botella, ralentizando el sistema.

Por ello, todas las computadoras hoy en día implementan controladores DMA, los cuales los configura el procesador con los siguientes datos:

- dirección física base de memoria en la cual operará
- cantidad de datos a transferir
- el puerto del dispositivo
- el sentido en que se efectuará la operación (del dispositivo a memoria o de memoria al dispositivo)

Una vez configurado, el DMA toma el control de la transferencia de datos, y lo realiza sin involucrar al procesador, generando una interrupción al procesador sólo cuando toda la transferencia de datos ha concluido o si ocurre algún error.

2.4. Llamadas al sistema

Cuando un proceso requiere de alguna acción privilegiada, accede a ellas realizando una llamada al sistema. Pueden agruparse en: control de procesos, manipulación de dispositivos, manipulación de archivos, mantenimiento de la información, comunicaciones, o protección.

3. Administración de procesos

Un proceso es la imagen en memoria de un programa, junto con la información relacionada con el estado de su ejecución. Decimos que mientras que un programa es una entidad pasiva, un proceso es una entidad activa. Algo a destacar, es que si bien el sistema brinda la ilusión de que se están ejecutando muchos procesos al mismo tiempo, la mayor parte de ellos está esperando su turno. Es sabido que en un momento dado, puede haber a lo sumo ejecutándose un número de procesos igual a la cantidad de procesadores.

La información del proceso estará en un PCB, que incluye los campos: id del proceso, estado, contador de programa, registros del CPU, prioridad del proceso, información de la memoria del mismo, tabla de archivos I/O.

3.1. Estados de un proceso

- Nuevo: se crea el proceso y se le están asignando los recursos.
- Listo: está esperando un procesador.
- En ejecución: está siendo procesado en este momento.

- Bloqueado: está bloqueado a la espera de algún evento (I/O, Señal, etc).
- Zombie: Terminó su ejecución, pero el SO debe realizar algunas limpiezas antes de poder borrarlo.
- Terminado: completó las tareas, y es borrado de la tabla de procesos.

3.2. Diferencias con hilos

Todos los hilos de un proceso comparten un sólo espacio de direccionamiento en memoria y los archivos y dispositivos abiertos. Cada uno de los hilos se ejecuta de forma secuencial y maneja su propio contador de programa y pila.

Los procesos que implementan hilos ganan un poco en el rendimiento gracias a no tener que reemplazar el PCB activo cuando intercalan la ejecución de sus diferentes hilos, y ganan mucho más por la ventaja de compartir memoria (los sistemas de mensajes disponibles). Cualquier llamada al sistema, bloqueante, interrumpirá la ejecución de todos los hilos, dado que el control pasa al SO, quien NO conoce nada sobre los hilos del proceso.

Existe otro tipo de hilos, que si son visibles al SO, que son los hilos del kernel. Los mismos aprovechan la comunicación con el SO tanto para solicitudes de recursos como para una gestión de recursos más comparable con una situación de multiproceso estándar. Este es el caso de los pthreads.

Existen tres patrones de trabajos entre hilos:

- Jefe/trabajador: Hay un hilo jefe que genera o recopila tareas para realizar, las separa y se las entrega a los hilos trabajadores. Generalmente cuando los hilos trabajadores finalizan su tarea, notifican al jefe y son eliminados. Ejemplo de uso: servidores.
- Equipo de trabajo: Se crean muchos hilos idénticos que realizarán las mismas tareas sobre diferentes datos. En este caso, los hilos no terminan si no que son sincronizados. Ejemplo de uso: cálculos matemáticos.
- Línea de ensamblado: Los hilos trabajan de forma secuencial en distintos pasos de una misma tarea, pero pueden estar ejecutándose paralelamente si utilizan información distinta. En este caso, todos los hilos son distintos.

3.3. Comunicación entre procesos: Señales

Una señal es una notificación asíncrona enviada a un proceso o hilo en específico dentro del mismo proceso para notificar un evento, utilizando distintos códigos según el tipo. El SO interrumpe el flujo de ejecución para enviar una señal, y la misma interrupción se realiza durante cualquier instrucción no atómica. Una gran ventaja de las señales, es que son parte de un mecanismo liviano computacionalmente.

Funciones utilizadas para trabajar con señales:

- kill: envía la señal especificada al proceso dado, retorna 0 o -1.
  `int kill (pid_t pid, int sig)`
- getpid: retorna el process ID del proceso llamante.
  `pid_t getpid(void);`
- raise: autoenvía una señal.
  `int raise (int sig)`
- signal: define un manejador para una señal, ya sea con una función handler que será invocada cuando ocurra la señal, o bien con SIG_IGN o SIG_DFL.
  `sighandler_t signal(int signum, sighandler_t handler)`.
- sigaction: utilizada al igual que signal, pero con la ventaja de poder proveer más información.
  `int sigaction(int signum, struct sigaction *act, struct sigaction *oldact)`, donde sigaction es una estructura que tiene dos tipos de handler, una mascara, unas banderas y una función para restaurar.
- sigprocmask: modifica el conjunto de señales bloqueadas para el proceso (la signal mask), es utilizada para controlar problemas de concurrencia.
- alarm: programa un temporizador y cuando llega a cero envía la señal SIGALRM

Los comportamientos definidos por defecto al atraparse una señal pueden ser: terminar el programa (SIGKILL, SIGINT, SIGALRM), terminarlo anormalmente (SIGSEGV, SIGFPE), ignorar la señal (SIGCHLD), y detener (SIGSTOP, SIGTSTP) o continuar el programa (SIGCONT). Las señales que terminan un proceso, cambian su estado de listo, running, o bloqueado, a zombie. Por otro lado, las señales que lo detienen, cambian su estado de listo o running a bloqueado, mientras que las señales que continuan lo cambian de bloqueado a listo.

4. Planificación de procesos

Se refiere a cómo determina el SO el orden en que irá cediendo el uso del CPU a los procesos que lo vayan solicitando y a las políticas que utilizará para determinarlo. Hay tres tipos de planificación:

- **A corto plazo**: Decide decenas de veces por segundo planificar cuál de los procesos que están listos para ejecutarse, utilizará el CPU. Este planificador se denomina `dispatcher`. Hace los pasos entre estados **listo - en ejecución**

- **A mediano plazo**: Decide cuáles procesos es conveniente bloquear en determinado momento, sea por escasez de recursos o porque están realizando alguna solicitud que no puede satisfacerse inmediatamente. Este planificador se denomina `scheduler`. Hace los pasos entre estados **bloqueado - listo** y **en ejecución - bloqueado**
- **A largo plazo**: Decide qué procesos serán los siguientes en ser iniciados. En los sistemas de uso interactivo no se utiliza pues es algo que decide el usuario. Hace los pasos de estado **nuevo - listo**.

4.1. Procesos cortos vs largos

En todo momento, los procesos que están en ejecución y listos, pueden separarse en procesos largos o cortos, según cuanto tiempo de CPU necesitan. Los procesos largos son aquellos que por mucho tiempo han estado listos o en ejecución, en cambio los cortos suelen pasan mucho tiempo bloqueados esperando eventos (como es el caso de los interactivos). Por lo general, se busca dar un tratamiento preferente a los procesos cortos.

4.2. Parámetros de respuesta

Para permitir que una misma implementación pueda sobrevivir a la evolución del hardware, se miden con ticks y quantums. Como mencionamos anteriormente, los ticks son una fracción de tiempo durante la cual se puede realizar trabajo útil, esto es, usar el CPU sin interrupción. Se determinan por señales emitidas por un temporizador. Por otro lado, un quantum es el tiempo mínimo que se permitirá a un proceso el uso del procesador, implementado como una cantidad determinada de ticks. Definimos entonces para un proceso p y un tiempo t de ejecución:

- **Tiempo de respuesta** (T): es el tiempo total. Se obtiene contando desde que el proceso llega a la cola de listos. Teniendo un único procesador y un único proceso, $T = t$.
- **Tiempo en espera** (E = T - t): es el tiempo perdido, el tiempo que está listo pero esperando, se desearía que sea cero.
- Proporción de **penalización** (P = T/t): qué tan penalizado estuvo el proceso.
- Proporción de **respuesta** (R = t/T): cuánto porcentaje del tiempo obtuvo el procesador (pudo ejecutar).
- α = frecuencia de llegada: cuántos procesos entran en un rango definido.
- β = tiempo de servicio requerido: cuántos procesos terminan en un rango definido.

- Valor de **saturación** $= \alpha/\beta$. Si es 0, nunca llegan procesos nuevos y si es 1, los procesos se despachan al mismo ritmo que llegan. Si es mayor a 1, llegan más procesos que los que puede darle respuesta el SO, y la cola de procesos listos tiende a crecer. Gracias a este valor, se pueden hacer análisis sobre que políticas son convenientes en cada caso.

También consideramos:

- **Tiempo real**: es el tiempo total de inicio a fin, es mayor que el tiempo de CPU.
- **Tiempo núcleo**: el sistema está decidiendo políticas de planificación o haciendo cambios de contexto, o realizando swap en disco, y no se considera como tiempo de CPU del proceso. En espacio núcleo, diríamos que es el tiempo de las tareas no asociadas a un proceso en particular.
- **Tiempo de sistema**: es el tiempo que pasa un proceso en espacio núcleo (por ejemplo atendiendo llamadas al sistema), sí se cuenta en el tiempo del proceso.
- **Tiempo de usuario**: tiempo que pasa el proceso en modo usuario
- **Tiempo de uso del procesador**: tiempo en el cual se uso la CPU para un proceso ya sea en modo usuario o núcleo (suma de los dos anteriores).
- **Tiempo idle**: tiempo en el cual el CPU no realiza ningún trabajo, la cola de procesos está vacía.
- **Utilización del CPU**: porcentaje del tiempo en el cual el CPU realiza trabajo útil, está relacionado a cuánto tiempo pasa en idle.

4.3. Algoritmos de planificación

La tarea principal del planificador es, entre todos los procesos listos, elegir cuál es el siguiente a ejecutar. Objetivos del planificador:

1. Ser justa: todas las compilaciones de programas relativamente similares o bien procesos que necesitan los mismos recursos o el mismo tiempo, deberían tardar lo mismo.
2. Ser predecible: debe ser posible aproximar cuánto va a tardar un proceso, antes de lanzarlo, incluso con distintas cargas en el sistema.
3. Maximizar el rendimiento: que el CPU esté en uso la mayor parte del tiempo.
4. Minimizar la sobrecarga: realizar decisiones rápidas, evitar cambios de contexto innecesarios o análisis largos.
5. Equilibrar el uso de recursos: se le da prioridad a procesos que utilizan menos recursos sobre los que los utilizan demasiado.

6. Evitar la postergación indefinida: evitar que los procesos caigan en inanición.
7. Favorecer el uso esperado del sistema: proveerle mejor respuesta a los procesos interactivos.
8. Dar preferencia a los procesos que podrían causar bloqueos.
9. Dar preferencia a los procesos con un comportamiento estable.
10. Degradarse suavemente: a medida que aumentan los procesos, es necesario lograr que los ya aceptados no sean muy afectados por los nuevos.

Es importante destacar que la planificación tiene varios objetivos, pero muchos de ellos son contradictorios. Por ejemplo, priorizar una decisión justa en cuanto al uso del CPU, lleva a mucha carga burocrática/administrativa y no permite minimizar el uso de recursos del CPU.

4.3.1. FCFS

Simplemente se ejecutan los procesos en el orden en el que llegan, y se les permite utilizar el CPU hasta que sueltan el control. El despachador es una cola FIFO. Características:

- Minimiza la cantidad de cambios de contexto.
- Reduce la sobrecarga administrativa (seleccionar el siguiente es O(1)).
- No requiere hardware de apoyo (como un temporizador).
- Favorece los procesos largos.
- No utiliza información intrínseca.
- Los últimos en llegar tienen un rendimiento inaceptable.
- Es inadecuado para trabajos interactivos (no puedo asignar prioridades).
- Multitarea **cooperativa**.

4.3.2. Round Robin

Intenta dar una respuesta buena tanto para procesos cortos como largos. Garantiza además una división justa del CPU. Les asigna a los procesos quantums, y si un proceso no termina luego de un quantum (q), se agrega al final de la cola. Además, si un proceso es despertado por otro planificador, se agrega al final de la cola. Características:

- Multitarea **apropiativa**, utilizando quantums.

- Mientras más se incrementa q, más tiende a ser FIFO.
- Mientras más se decrementa q, da una mayor ilusión de un procesador dedicado, pero a la vez más lento por la sobrecarga administrativa.
- Globalmente, el rendimiento es peor que el de FCFS.
- Aumentar q mejora el tiempo de respuesta promedio, pero penaliza a los procesos cortos.
- No utiliza información intrínseca, ni permite asignar prioridades.

4.3.3. SPN

Necesita información en anticipado de cuánto tiempo necesita cada proceso, y en ese caso es una planificación que intenta ser más justa. Como generalmente no puede saber de manera cierta el tiempo del proceso, se fija en el historial si tiende a ser un proceso corto o largo. Características:

- Multitarea **cooperativa**.
- Define un promedio exponencial con un factor atenuante $e_p = f_{e_p} + q(1 - f_{e_p})$ donde $0 < f_{e_p} < 1$ y q es la cantida de quantums que utilizó en su última ejecución.
- Favorece a los procesos cortos. Un proceso muy largo va a sufrir inanición.
- Utiliza información intrínseca.

4.3.4. SPN apropiativo

Comportamiento similar al SPN en la mayoría de los casos. Al despachar primero los procesos más cortos, mantiene la lista de procesos corta, lo que termina en menores índices de penalización.

- Implementa multitarea **apropiativa**.
- Utiliza información intrínseca.

4.3.5. HRPN

Elige el proceso más penalizado a continuación. Intenta no favorecer a los largos, como es el caso de FCFS, ni a los cortos como SPN, busca un equilibrio entre ellos. Todo proceso inicia con $P = 1$. Cada vez que espera un tiempo ω, se actualiza $P = \omega + t/t$. Y se elige como próximo al proceso con mayor P. Características:

- Si $\rho < 1$ evitará que incluso los más largos sufran inanición.
- Es muy costoso calcular P cada vez que se cambia de proceso.
- Implementa multitarea **cooperativa**.
- Utiliza información intrínseca.

4.3.6. SRR

Busca favorecer a los procesos que ya hayan pasado tiempo ejecutando antes que a los nuevos. Los recién llegados no son programados para su ejecución, si no que se agregan a una cola de procesos nuevos. El resto se encuentra en una cola de procesos aceptados. Cuando un proceso nuevo alcanza la prioridad de uno aceptado, pasa a ser aceptado. Además, cuando la cola de aceptados es vacía, se acepta el proceso nuevo con mayor prioridad. Características:

- Implementa multitarea **apropiativa**.
- a indica el ritmo con el que se incrementa la prioridad de los nuevos.
- b indica el ritmo con el que se incrementa la prioridad de los aceptados.
- $b/a < 1$ todo proceso nuevo alcanzará en algún momento los aceptados. Es relativamente egoísta, porque los nuevos no entran directamente. Este es el caso más normal.
- $b/a > 1$ se convierte en FCFS.
- $b/a = 0$ se convierte en RR, se aceptan mientras llegan.
- Utiliza información intrínseca.

4.3.7. FB

Se define como retroalimentación multinivel. Utiliza varias colas, se emplea mucho hoy en día. Cuando hay más de una cola como en este caso, tienen un orden preestablecido, y el planificador eligirá entre los que estén en la de mayor prioridad. Sólo cuando no haya, irá a la siguiente cola. Tras un número determinado de operaciones en un proceso, lo degrada a la siguiente cola. Características:

- Todas las colas tienen el mismo tratamiento general.
- Favorece a los procesos cortos.
- Implementa multitarea **cooperativa**, utilizando quantums.

- Se puede ajustar la cantidad de ejecuciones de un proceso antes de degradarlo, y la duración de q.
- Con los ajustes necesarios, los promedios resultan mejores todavía que RR.
- También puede realizarse aging para evitar inanición de procesos largos, incrementándolos de prioridad cuando hace mucho tiempo no se ejecutan.
- Utiliza información intrínseca.

4.3.8. Lotería

Cada proceso tiene un número determinado de boletos, y cada boleto representa una posibilidad de jugar a la lotería. Cuando debe decidirse el próximo a ejecutar, el planificador elige un boleto al azar.

- Un proceso que tiene más prioridad, simplemente tendrá más boletos.
- Los procesos pueden cooperar, es decir, transferirse boletos.
- Es justo, tanto para procesos cortos como largos.
- Implementa multitarea **cooperativa**, utilizando quantums.
- No utiliza información intrínseca.

4.3.9. Métodos dependientes del sistema

Algunas características a tener en cuenta para decidir si una política de planificación es adecuada son:

- Si los procesos listos son en promedio cortos, y el valor $\rho < 1$, se deberían utilizar los métodos que tengan menos sobrecarga administrativa. Estos son, FCFS, SPN o RR con q grande.
- Si el despachador observa que la cola de tareas es muy larga, debe utilizar un RR con q chico, o PSPN, los cuales garantizan buena distribución de la atención.
- Siempre es una buena opción utilizar un esquema simple de ronda, donde el q puede cambiarse periódicamente, dependiendo de la cantidad de procesos. Si hay pocos procesos, se reduce la cantidad de cambios de contexto aumentando q. Otra opción es fijar q para el proceso dependiendo su prioridad.

4.4. Planificación de hilos

Primero, consideraremos tres modelos principales de mapeo de hilos:

- Muchos a uno: los hilos de usuario entran en este caso. Para el SO, hay un sólo proceso, y el mismo es el que se encarga de repartir el tiempo entre sus hilos cuando está ejecutando. Este caso no aprovecha realmente el paralelismo.
- Uno a uno: Cada hilo es ejecutado como un proceso ligero. Se aprovechan las ventajas del paralelismo, pudiendo ejecutarse cada hilo en un procesador distinto.
- Muchos a muchos: es un mecanismo que permite hilos de ambos modelos.

4.5. Planificación de multiprocesadores

Una primer decisión para analizar, es si se quiere que haya una única cola de procesos listos que sea global, o que cada procesador tenga su propia lista de procesos pendientes. Si la cola es global, no es necesario realizar balanceo de cargas o tener en cuenta la afinidad del proceso. Sin embargo, este mecanismo no es utilizado puesto que conlleva una gran pérdida de flexibilidad del sistema.

Por eso, en las siguientes secciones, tendremos en cuenta que cada procesador tiene su propia cola de tareas.

4.5.1. Afinidad

En un entorno multiprocesador, después de que un proceso se ejecutó por cierto tiempo, tendrá parte importante de sus datos en el caché del procesador en el que fue ejecutado, y esto idealmente sería aprovechado en la próxima ejecución.

Decimos que un proceso tiene `afinidad` por un procesador, cuando se desea que se vuelva a ejecutar en él. Si este deseo es simplemente una preferencia, pero pueden ocurrir determinados patrones que lleven su ejecución a otro procesador, se dice que la afinidad es `suave`. En cambio, si está garantizado que volverá a ejecutarse en el mismo procesador, la afinidad es `dura`.

4.5.2. Balanceo de cargas

Si consideramos la situación ideal en un sistema multiprocesador, querremos que todos los CPU's estén despachando trabajos al 100 % de su capacidad. Sin embargo, esto es sólo teórico.

De algo que sí estamos seguros es que la divergencia entre la carga de cada uno, debe ser lo más pequeña posible y para lograr esto se emplean esquemas de balanceo de cargas. Estos son algoritmos que analizan los estados de las colas, y de ser necesario realizan migraciones

para transferir procesos de una cola que tiene un gran valor de saturación a otra que este menos cargada.

Por un lado, tenemos `la migración activa` en la cual una tarea que ejecuta como parte del núcleo revisa periódicamente el estado de los procesadores y transfiere procesos entre ellos según sea necesario. Por otro lado, en la `migración pasiva`, recién cuando un procesador queda sin tareas ejecuta un proceso especial (*idle*), el cual significa que ese tiempo puede ser utilizado para ejecutar tareas de núcleo. Entre ellas, suele verificar que haya procesos en colas de otros CPU's y trasladárselos a él mismo.

Ambos mecanismos son utilizados en los procesadores modernos, y conllevan una penalización por las afinidades de los CPU's.

4.6. Inversión de prioridades

Un efecto colateral de que las estructuras del núcleo estén protegidas por mecanismos de sincronización, es que puede presentarse la inversión de prioridades.

¿Qué es? Es la situación en donde un proceso que necesita acceder a un recurso, debe esperar porque está siendo usado por un proceso de menor prioridad. Este problema puede minimizarse aplicando diferentes mecanismos.

- **Techo de prioridad:** Designa una prioridad máxima al recurso compartido, la cual es la prioridad más alta de entre cualquier proceso que pueda acceder a ese recurso. Cuando un proceso adquiere un recurso, su prioridad se eleva a la prioridad máxima del mismo y cuando libera el recurso, su prioridad se restaura a su valor original.
- **Herencia de prioridades:** Se aumenta temporalmente la prioridad de la tarea de menor prioridad que contiene el recurso que necesita la tarea de mayor prioridad. De esta manera, la tarea que contiene el recurso, está heredando la prioridad de la que lo necesita y con ello puede finalizar más rápido y no perjudicarla.

5. Linux Scheduler

Los objetivos del planificador son dos:

- Poder adaptarse lo mejor posible a cualquier entorno en el cual se utilice el sistema operativo.
- Optimizar el rendimiento de los procesadores sin dejar de lado a las tareas interactivas.

La idea general de un planificador es intentar llevar a la CPU a una multitarea ideal y precisa, es decir, lograr una equidad entre tareas. Sin embargo, este es un concepto teórico que especifica que dado un cierto poder de computo, cuando haya varias tareas ejecutándose, cada una reciba una porción equitativa del total y nada sea desperdiciado.

5.1. Tareas

Los tipos de tareas son dos: Batch e Interactivas, las cuales tienen su correlación con largas y cortas respectivamente. Las primeras son aquellas que realizan un uso intensivo del CPU, y son las que se permite que su proceso demore un poco. Por otro lado, las tareas interactivas son aquellas que están en estado bloqueado la mayoría del tiempo, esperando la ocurrencia de algún evento determinado. Lo ideal es que cuando el evento ocurra estas tareas reaccionen de inmediato. Son tareas que no necesariamente interactúan con el usuario, aunque es lo más típico. Otro ejemplo sería accesos al disco.

Por otro lado, podemos observar diferentes estados en una tarea, los mismos son:

- Task Running: ya sea lista o en ejecución.
- Task Interruptable: bloqueada, con la posibilidad de retirársele recursos.
- Task Uninterruptable: bloqueada, sin la posibilidad de retírársele recursos, típicamente por una llamada a memoria.
- Task Stopped: bloqueada por una señal.
- Task Traced: bloqueada por estar siendo depurada.
- Task Zombie.
- Exit Dead: terminada.

Ahora, podemos definir que la función del planificador es ocuparse de las tareas en estado TASK RUNNING. Por otro lado, vemos una segunda clasificación de tareas:

- Tareas en tiempo real: son aquellas que requieren atención y tiempos de respuesta cortos en relación a la velocidad del procesador. Para ellas, se encuentran dos tipos de planificación: SCHED_FIFO y SCHED_RR. Las tareas afectadas por la clase SCHED_FIFO sólo pueden ser reemplazadas por tareas de mayor prioridad, sino se ejecutan hasta cambiar a estado bloqueada o terminar. En cambio, si están afectadas por la clase SCHED_RR utilizan política de RR con un quantum fijo.
- Tareas normales: son el resto de las tareas, y se tratan como la clase SCHED_OTHER.

5.2. Prioridades

Cada tarea, tiene asignada una prioridad estática y una dinámica. La prioridad estática, puede ajustarse antes de comenzar a ejecutarse el programa usando las funciones `nice` y `set_priority`. Además, es heredada por hijos o hilos del proceso. Por otro lado, la dinámica es una prioridad interna del planificador (asignada por el mismo), y puede cambiarla según

le sea conveniente, aunque se basa en la prioridad estática inicialmente. Para tener una idea general, observamos que en Linux se utilizan prioridades entre 0 y 139, siendo las primeras 100 para tareas en tiempo real y el resto (100-139) para tareas normales.

5.3. Historia de los planificadores

Al comienzo, en el Kernel 1.2, se utilizó RR, y luego en el Kernel 2.2 apareció el concepto de clases de planificación (tiempo real, normales), y se incorporó soporte para los multiprocesadores.

Recién en el Kernel 2.4 se implementa algo nuevo, un planificador basado en épocas. Durante una época todas las tareas reciben atención, algunas más dependiendo de su prioridad. Además, una tarea que no consuma todo su lapso de tiempo, era beneficiada en la siguiente época. Sin embargo, su problema era que la elección de la próxima tarea era de orden O(n), donde n es el número de tareas listas.

En el mismo kernel, terminó incorporándose un nuevo planificador, el scheduler O(1), en el cual la elección de la próxima tarea se realizaba en tiempo constante. En él se utilizaban dos arreglos de colas, uno de tareas activas y otro de tareas expiradas, y cuando las tareas reciben su tiempo de CPU, se pasan a la cola de tareas expiradas. Si la cola de activas es vacía, simplemente se intercambian los arreglos. Además, las tareas interactivas reciben un bonus de prioridad mientras que las tareas batch son penalizadas decrementando su prioridad. Este procesador dejó de utilizarse debido a las complicaciones para manejar las prioridades.

Este recorrido en distintos planificadores finalizó con el CFS: *Completely Fair Scheduler*, el cual le agrega un parche al kernel del scheduler. Lo que pretende CFS es simular el comportamiento de esta CPU ideal: dividir el poder de cómputo equitativamente entre todas las tareas listas. Pero en una CPU real no pueden ejecutarse dos tareas a la vez, así que se opta por intentar repartir el tiempo de uso de la CPU.

5.4. CPU ideal

Para simular una CPU ideal, se usa un `fair clock`. La frecuencia de incremento del mismo se calcula dividiendo el tiempo real por el número de tareas, y representa el poder de cómputo asignado a una tarea. Cada tarea tiene un `wait_runtime` asociado, que se incrementa cuando está esperando y se decrementa cuando está corriendo, es decir, es el tiempo que pasó esperando la CPU. Luego, el cambio de contexto se realiza cuando una tarea que no está corriendo tiene un tiempo mayor que la que se está ejecutando en ese momento.

Para tener las tareas ordenadas, se utiliza un RBT, que tiene trabajo de orden O(lg n), y se toma el nodo más a la izquierda como próximo a ejecutar. Además, se asignan prioridades asociando un peso a cada prioridad, de forma que las tareas con menor prioridad ven pasar el tiempo más rápido.

En el kernel 2.6.24, se hizo una reestructuración del CFS en donde se utilizaba un tiempo

virtual `vruntime`. El vruntime se incrementa con el reloj real (con un factor relacionado a la prioridad) y la tarea con menor vruntime es entonces la indicada para tomar el procesador. Se usa igualmente un RBT, pero esta vez ordenado por vruntime. Además en esta versión se agregó "group scheduling" y "multiprogramming".

La `planificación de grupos` es fundamental para lograr que el planificador sea equitativo entre procesos. Esto es, se asignan tareas por usuario o según la jerarquía de procesos para evitar por ejemplo casos en los que un usuario utilice mucho más tiempo el procesador que otro debido a que lanzó más tareas. Lo más importante de este concepto es saber que ahora el planificador tiene la posibilidad de agrupar.

En el caso de `equipos multiprocesadores`, cada procesador tiene su propia cola de procesos listos, representada por el RBT de CFS, junto con una medida de su carga. Luego, comparando dichas medidas puede realizarse una migración para quitar una tarea de un procesador y pasarla a otro menos cargado. Sin embargo, esto conlleva una incidencia negativa en la eficiencia de los cachés, por lo que se realiza únicamente en casos extremos.

Por último, es importante recalcar el nuevo concepto de `scheduling domains` que permiten agrupar uno o más procesadores jerárquicamente para realizar balances de carga. Cada dominio tiene su propio grupo de CPU's que comparten propiedades y políticas, y los trata como uno único. Luego, el balanceo de cargas se realiza entre grupos, y el planificador no se preocupa por lo que ocurre dentro de uno.

6. Administración de memoria

6.1. Funciones y operaciones

La memoria de la computadora se puede dividir en una jerarquía:

`Procesador (Registros) - Primaria (RAM) - Secundaria (Almacenamiento)`.

En dicha jerarquía, a más alto nivel la memoria es más rápida pero más costosa y por tanto de menor tamaño. El único espacio de almacenamiento que el procesador puede utilizar directamente, más allá de los registros, es la RAM. Todos los programas a ejecutar deben cargarse a la memoria del sistema antes de ser utilizados. Es por eso que el SO debe administrar la memoria para permitir que varios procesos la compartan, con la ayuda de hardware especializado (MMU).

Como sabemos, la memoria está estructurada como un arreglo direccionable de bytes. Esto significa que al solicitar el contenido de una dirección específica, se entregará un byte, y no menos. Además, tiene el mismo costo en cuanto a transferencia, enviar 1 u 8 bytes. Decimos entonces que un procesador soporta un espacio de direccionamiento de n *bits* si puede referirse directamente a hasta 2^n bytes.

En la mayoría d los sistemas modernos hay muchos procesos a la vez en memoria. Más allá de las ventajas que esto trae, genera un problema: ¿Cómo se logra que muchos procesos usen la

RAM sin complicaciones? La idea es que cada uno use una porción separada, que no interfiera con la de los demás. Así se puede compartir la RAM entre los procesos.

Sin embargo, a medida que los SO y los programas crecieron en tamaño, el tamaño de la RAM (más aún al ser compartida) quedó chico. Fue necesario entonces abstraer el espacio de almacenamiento para dar la ilusión de contar con más memoria de la que está directamente disponible.

Por ello, en el medio entre el CPU y la RAM, se añadió la MMU; (muchas veces se encuentra dentro del chip, pero conceptualmente es una unidad separada del procesador). El mecanismo es el siguiente: La CPU pone en el bus la dirección de la siguiente instrucción, la MMU la recibe y realiza un procesamiento (independiente de la ALU y los registros) y la traduce.

Con asistencia de la MMU, es posible configurar un espacio lineal contiguo para cada proceso, y para el mismo SO, que se proyecta a memoria física y a un almacenamiento secundario. La MMU es también la encargada de verificar que un proceso no tenga acceso a leer o modificar datos de otro. Una primera aproximación a la protección de acceso se implementa usando un registro base y un registro límite, con ellos la MMU puede comparar sin penalidad cada acceso a memoria para verificar que esté en el rango permitido.

Por otro lado, conforme el procesador avanza en la ejecución de las instrucciones, se producen accesos a memoria. Para evitar que el procesador detenga la ejecución cuando solicita un contenido de memoria que no está disponible todavía, se implementó la memoria caché. Como el CPU no puede, a diferencia del SO, realizar cambios de contexto porque la lista de ejecución es estrictamente secuencial, se ideó una solución que involucra tener una memoria de alta velocidad entre la RAM y el CPU que guarda copias de las páginas que van siendo accedidas. Aplicando el principio de localidad de referencia, cuando el procesador solicita al hardware una dirección de memoria, el hardware transfiere un bloque o página completo. Este principio se basa en tres localidades distintas:

- **Localidad temporal**: Es probable que un recurso que fue empleado recientemente vuelva a emplearse en un futuro cercano.
- **Localidad espacial**: La probabilidad de que un recurso aún no requerido sea accedido es mucho mayor si fue requerido algún recurso cercano.
- **Localidad secuencial**: Un recurso (particularmente la memoria) tiende a ser requerido de forma secuencial.

Cuando un SO inicia un proceso, le proporciona una estructura para que vaya guardando la información. La misma es:

- **Segmento de texto**: Es la imagen en memoria de las instrucciones. Usualmente, ocupa las direcciones más bajas del espacio en memoria.
- **Sección de datos**: Espacio fijo pre-asignado para las variables globales y datos inicializados. Fijado en tiempo de compilación.

- **Heap**: Espacio que se emplea para la asignación dinámica de memoria durante la ejecución del proceso. Se ubica por encima de la sección de datos, y crece hacia arriba.
- **Stack**: Se usa para almacenar la secuencia de funciones que han sido llamadas dentro del proceso, con sus parámetros, direcciones de retorno, variables locales, etc. Ocupa la parte más alta del espacio en memoria, y crece hacia abajo.

6.2. Asignación de memoria contigua

6.2.1. Partición de la memoria y fragmentación

Cuando comenzó a ser necesario resolver cómo asignar el espacio en memoria a diferentes procesos, la primer respuesta fue fragmentar la memoria. Esto es, asignar a cada programa a ser ejecutado un bloque contiguo de memoria de un tamaño fijo. En este caso, a medida que más procesos van siendo ejecutados, comienza el fenómeno de la fragmentación. Esto es, aparecen regiones de memoria disponible, interrumpidas por regiones de memoria usadas por los procesos que aún se encuentran activos (**fragmentación externa**). Por otro lado, como los bloques son de tamaño fijo, parte de lo asignado a un proceso puede ser desperdiciado (**fragmentación interna**). El problema de la fragmentación es crucial, pues por cada N bloques asignados, se perderán del orden de $0.5N$ bloques por fragmentación.

Al crear un nuevo proceso, el SO tiene 3 estrategias posibles:

- **Primer ajuste**: Se toma el primer bloque con el tamaño suficiente para alojar el nuevo proceso. Es el mecanismo más simple de implementar y el de más rápida ejecución, pero puede causar gran desperdicio de memoria.
- **Mejor ajuste**: Se busca entre todos los bloques disponibles cuál es el que mejor se ajusta al tamaño requerido por el nuevo proceso. Es un mecanismo con un costo muy alto, pero tiene un buen resultado en cuanto a utilización de recursos.
- **Peor ajuste**: Se busca cuál es el bloque más grande disponible. Esta operación puede ser incluso más rápida que la de primer ajuste, e intenta que los bloques que queden sean tan grandes como sea posible.

6.2.2. Compactación

En el caso de que el espacio total libre sea suficiente para almacenar un nuevo proceso, pero el mismo esté fragmentado en distintas partes de la memoria, se puede implementar la compactación. Sin embargo, esta estrategia solo puede utilizarse si los procesos emplean resolución de direcciones en tiempo de compilación. En este caso, la compactación consiste en "juntar"todos los bloques de memoria utilizados y permitir que los bloques libres queden contiguos. Claramente, estamos moviendo toda la memoria por lo cual este mecanismo tiene un costo muy alto.

6.2.3. Swapping

Es una estrategia bastante útil en lo que se refiere a particionar la memoria. Lo que se realiza es permitirle al SO comprometer más memoria de la que tiene físicamente disponible para el proceso. Cuando la memoria se acaba, el sistema suspende un proceso (usualmente uno bloqueado) y almacena una copia de su imagen en memoria en almacenamiento secundario para luego poder restaurarlo.

6.3. Segmentación

La segmentación es un concepto que se aplica directamente a la arquitectura del procesador. Permite separar las regiones de la memoria en segmentos, cada uno de los cuales puede tener diferentes permisos de acceso. El SO puede indicar, por ejemplo, que un segmento sea solo de lectura y escritura, mientras que otro puede no tener permisos de ejecución. Esta es una de sus principales ventajas. Cuando se implementa segmentación, siempre se utilizan referencias a la memoria en un espacio segmentado, llamadas `direcciones lógicas`. Las mismas están formadas por un selector de segmento y un desplazamiento. Para realizar la traducción a `direcciones físicas`, la MMU toma el selector, obtiene la dirección base, el tamaño del segmento y sus atributos de protección. Luego, le suma a la dirección base el desplazamiento, obteniendo la dirección deseada.

6.3.1. Excepciones

La traducción de una dirección lógica a una dirección física puede fallar por diferentes razones: si el segmento no se encuentra en memoria, ocurrirá una excepción del tipo `segmento no presente`. Por otro lado, si el desplazamiento especificado es mayor al tamaño del segmento, ocurrirá una excepción del tipo `violación de segmento`. Si el tipo de acceso pedido no coincide con los permisos, ocurrirá una `violación de seguridad`, mientras que si el segmento no existe, devolverá una excepción de `segmento inválido`. En caso de haber más de una excepción, el sistema reaccionará primero a la más severa.

El principal problema de la segmentación son los espacios libres que quedan entre los segmentos cuando los procesos van terminando. Como se mencionó anteriormente, juntar los procesos (compactar) le sale bastante caro al SO. Para evitar hacer esto, se usa la paginación.

6.4. Paginación

Consiste en que cada proceso está dividido en varios bloques de tamaño fijo llamados páginas. Su principal beneficio es que no requiere que la asignación de memoria sea de un área contigua, aunque necesita de mayor soporte de hardware y mayor información relacionada a cada proceso. Se establece un mapeo entre la dirección física o real y la dirección lógica que se le

presenta al proceso. Nuevamente, se está dando al proceso la ilusión de que la memoria es de su uso exclusivo.

Se divide la RAM en una serie de marcos del mismo tamaño, y el espacio para cada proceso se divide en una serie de páginas, que coinciden en tamaño con los marcos, siendo este una potencia de dos. Luego, se establece una tabla de páginas, con la cual la MMU realiza el mapeo correspondiente.

En una arquitectura que maneja paginación, las direcciones que ve el CPU no son absolutas, si no que se componen por un identificador de página y un desplazamiento. Se puede tomar la paginación como una suerte de resolución o traducción de direcciones en tiempo de ejecución, pero con una base distinta para cada una de las páginas.

6.4.1. Tamaño de la página

Gracias a la paginación, se resuelve el problema de fragmentación externa, pues la memoria asignada a un proceso podría decirse que está desordenada, ubicada en posiciones no contiguas. Sin embargo, el problema de la fragmentación sigue estando, puesto que las páginas y marcos tienen un tamaño fijo. Si se elige un tamaño de página más grande, en muchos casos los procesos desperdiciaran parte de la memoria que se le asigna. Por otro lado, si se achica demasiado el tamaño de la página, se genera demasiada sobrecarga administrativa. Además, mientras más chicas sean las páginas, más páginas tendrá cada proceso, lo cual hará que los PCB guarden más información, haciendo que los cambios de contexto sean más pesados.

6.4.2. Tabla de páginas y TLB

En un principio, se empleaba un conjunto especial de registros para representar la tabla de páginas. Esto era posible dado que eran sistemas de 16 bits, con 8 páginas posibles, de 8 KB *c/u*; tenía sentido dedicar un registro a cada una. En los sistemas actuales, esto resultaría claramente imposible, pues no alcanzaría la memoria de los registros. Además, cada proceso tiene su propia tabla de páginas, por lo que se necesitaría realizar grandes transferencias de información en cada cambio de contexto.

Es por esto que la tabla de páginas se almacena en memoria, y se guarda en registros especiales el puntero a la dirección donde comienza (*page table base register o PTBR*) y su tamaño *page table length register o PTLR*. Se permite entonces que el cambio de contexto implique solamente actualizar dichos registros y que el espacio para almacenar la tabla sea mayor.

Un primer problema sobre lo mencionado anteriormente, es que para resolver una dirección lógica hace falta consultar la tabla de páginas en memoria, por lo que el tiempo efectivo se duplica. La salida a este dilema fue incluir un caché, llamado `Buffer de traducción adelantada`. Este buffer, denominado TLB (*Translation Lookaside Buffer*), es un hash en memoria de alta velocidad, dentro de la MMU, lo cual permite búsquedas en tiempo constante.

La ventaja de utilizar este caché, es que cuando el procesador accede a memoria, si la página correspondiente está en el TLB, tiene la dirección física en el momento. Por otro lado, si no se encuentra en el TLB se lanza un `page fault`, se consulta la memoria principal, y se agrega al TLB la página recién accedida. Es importante destacar que por las propiedades de localidad de referencia, las probabilidades de que las regiones más empleadas de la memoria durante un momento específico de ejecución no generen fallos de TLB, son muy altas aún utilizando relativamente pocas entradas para el TLB (generalmente tiene un 99 % de acierto).

Además, otro problema de almacenar la tabla de páginas en memoria, es el tamaño de dicha tabla. Un ejemplo sería pensar que en una arquitectura de direcciones de 32 bits, con páginas de 4KB, la tabla de páginas de un proceso podría llegar a ocupar 4MB. Una solución "fácil" sería aumentar el tamaño de la página, pero como vimos anteriormente, esto aumentaría la fragmentación interna. Por eso se plantean soluciones más sofisticadas:

- **Paginación jerárquica**: La propia tabla de páginas está paginada; se forma una jerarquía de niveles de paginación. Al dividir la tabla en unidades más pequeñas, se evita tener que asignar todo de forma contigua en la memoria principal.
- **Tabla de páginas invertida**: Una sola tabla de páginas global para todos los procesos, que tiene una entrada por cada marco real de la memoria. La entrada k-ésima informa de la página lógica almacenada en el marco K y el proceso al que pertenece. Este mecanismo permite un gran ahorro en el espacio consumido por las tablas de páginas.
- **Tabla hash de páginas**.

6.5. Memoria compartida

Compartir áreas de memoria entre procesos puede proporcionar varios beneficios significativos en términos de eficiencia y uso de recursos. Por ejemplo, cuando diferentes procesos necesitan comunicarse entre sí, pueden utilizar áreas de memoria compartida en lugar de copiar datos entre ellos. Además, cuando un mismo programa se ejecuta varias veces, cada instancia generalmente utiliza las mismas instrucciones de código. Y sobre todo, este mecanismo es todavía más útil cuando se usan bibliotecas del sistema.

Para poder permitir la memoria compartida, el SO debe garantizar que las páginas correspondientes al código del programa sean de sólo lectura (se evita la sincronización entre procesos).

Como bien sabemos, el mecanismo más frecuentemente utilizado para crear un nuevo proceso es la llamada al sistema `fork()`. En este escenario, el SO crea un nuevo proceso idéntico al que realizó la syscall. El costo de `fork()` en Unix es muy bajo, y esto es gracias al sistema Copy On Write. Cuando se realiza la llamada a sistema, el SO simplemente crea las estructuras necesarias en la memoria del núcleo y dispone todas las páginas de memoria del proceso padre al hijo, para que sean compartidas entre ellos. Sin embargo, esto no será siempre así.

Las páginas de memoria de ambos procesos son las mismas mientras sean sólo leídas. Cuando un proceso modifique alguna página se le hará una copia de la misma para modificarla, y

dicha página dejará de ser compartida. Esto se puede lograr marcando todas las páginas compartidas como sólo lectura, con lo cual cuando un proceso intente modificar alguna página se generará un fallo. El SO, al notar que esto ocurre sobre un espacio CoW, en vez de responder al fallo terminando el proceso, copiará la página en donde se generó el fallo, y esta vez la marcará como RW. En resumen, el método CoW nos permite ahorrar un montón de memoria, y nos crea un poco de trabajo cuando se quiere escribir en esas páginas.

Puede surgir la duda de cómo diferencia el SO, cuando se intenta escribir en una página que tiene el bit de read-only encendido, si la página corresponde a .text o si únicamente es una página que está en CoW. Para poder marcar esta distinción, el SO lleva un rastreo de las páginas para saber si son de .text o no.

6.6. Memoria virtual

Para ofrecer a los procesos mayor espacio en memoria del que se cuenta físicamente, el sistema emplea espacio en almacenamiento secundario mediante un esquema de intercambio (swap) guardando y trayendo páginas enteras. Es importante apuntar que la memoria virtual es gestionada de forma automática y transparente por el SO.

Gracias a la MV, los programas pueden ejecutar como si tuvieran más memoria disponible de la que realmente hay, ya que sólo las partes necesarias del programa están en la RAM en un momento dado. En contraste con la segmentación, donde se permitía intercambio parcial de segmentos completos, la MV empleando paginación proporciona una manera más flexible y eficiente de manejar la memoria, pues se puede reemplazar página por página; se dice que aumenta la granularidad del swap.

6.6.1. Paginación sobre demanda

Cuando se comienza a hablar de memoria virtual, surge una idea muy interesante. Si bien a una computadora basada en la arquitectura von Neumann le es imposible ejecutar código que no esta cargado en memoria, éste sí puede comenzar a ejecutarse sin estar completamente cargado: basta con tener la página donde están las instrucciones que permiten continuar con su ejecución actual. Surge entonces la paginación sobre demanda: el concepto de leer únicamente la porción necesaria para comenzar la ejecución, e ir cargando las páginas a medida que son utilizadas.

Para implementar esta política, se agrega a la tabla de páginas un **bit de validez**, indicando para cada una si está o no presente en memoria. Si el proceso intenta utilizar una página que esté marcada como inválida, se genera un fallo de página, que lleva a que el SO lo suspenda y traiga a memoria la página solicitada para luego continuar con su ejecución.

1. Verifica en el PCB si esta solicitud corresponde a una página que ya ha sido asignada a este proceso.

2. En caso de que la referencia sea inválida, se termina el proceso.
3. Procede a traer la página del disco a la memoria. El primer paso es buscar un marco disponible.
4. Solicita al disco la lectura de la página en cuestión hacia el marco especificado.
5. Una vez que finaliza la lectura de disco, modifica tanto al PCB como al TLB para indicar que la página está en memoria.
6. Termina la suspensión del proceso, continuando con la instrucción que desencadenó al fallo. El proceso puede continuar sin notar que la página había sido intercambiada.

6.7. Algoritmos de reemplazo de páginas

Cuando se introducen los conceptos de memoria virtual al sistema, aparece el problema de sobre-comprometer memoria para administrar los procesos. Esto es, puede ocurrir que se necesiten cargar en memoria más páginas que los marcos disponibles. Para resolver esto, se implementó el espacio de intercambio.

Ahora, una página de un proceso puede estar almacenada en el disco. En la tabla de paginación, aparecerá como invalida y la MMU generará un fallo cuando el proceso intente utilizarla. Esto es lo que se conoce como `page fault`, lo cual puede resolver el SO en forma transparente para el proceso.

1. Busca en memoria una pagina libre y la marca como usada.
2. Busca en el disco la página y la guarda en la anterior conseguida.
3. Arregla la tabla de paginación, marca la entrada correspondiente válida y asigna el numero de página física.

Sin embargo, es bastante frecuente que habiendo poca memoria, o muchos procesos o mucho uso de buffers/caches, no haya páginas libres (cómo se supuso en (1)). En esos casos, el SO tiene que elegir qué pagina sacar de memoria (víctima) para poder colocar la nueva. Debemos escribirla en el disco y marcarla como inválida en la tabla de paginación correspondiente.

Claramente, esto implica una doble transferencia al disco, cuya velocidad es mucho menor, por lo que en grandes cantidades deteriora el rendimiento. Para mejorarlo, existe un bit de página sucia o modificación en la tabla de páginas (requiere apoyo de la MMU). Este bit se marca como apagado siempre que se carga una página a memoria, y es automáticamente encendido por hardware cuando se realiza un acceso de escritura a dicha página. Luego, si el SO eligió a la página como víctima, puede detectar que el bit está apagado, y se ahorra esa transferencia (la copia en disco de la página es igual que la que está en memoria).

6.7.1. Anomalía de Belady

Es un fenómeno interesante que se presenta con algunos algoritmos. Describe que, si bien la lógica indica que a mayor número de marcos disponibles se tendrá una menor cantidad de fallos, en algunos casos y bajo ciertos algoritmos, puede haber una regresión o degradación. Esto es, la cantidad de fallos aumenta aún más con una mayor cantidad de marcos. Por suerte, en simulaciones con características más cercanas a las de los patrones reales de los programas, su efecto observado es prácticamente nulo.

6.7.2. FIFO

Sencillamente se apunta a cuál fue la primer página que se cargó de las que están presentes en memoria, y se reemplaza cuando se debe agregar otra.

- Es el más simple, fácil implementación (lista enlazada circular).
- No se interesa en las últimas solicitudes, puede tener bajo rendimiento.
- No importan las frecuencias de uso de las páginas, todas tienen la misma probabilidad de ser reemplazadas.
- Vulnerable a la anomalía de Belady.

6.7.3. OPT, MIN

Es teórico, pero ofrece una cota interesante. Se elige como página víctima a aquella que no vaya a ser utilizada por un tiempo máximo. Requiere conocimiento de las necesidades a futuro del sistema, por lo que no es implementable.

- Ofrece una cota mínima. Permite ver qué tan cercano resulta otro algoritmo respecto al caso óptimo.
- Libre de la anomalía de Belady.

6.7.4. LRU

Intenta predecir cuándo será la próxima vez que se emplee cada una de las páginas que tiene en memoria, basandose en la historia reciente de su ejecución. Elige entonces como víctima la página que no ha sido empleada hace más tiempo.

Es complicada su implementación. Para el SO los fallos son fáciles de capturar, porque la MMU los genera y el SO se entera, pero en los aciertos la MMU no genera ninguna señal, y el SO no se entera. Por eso, se necesita apoyo de hardware (en la actualidad la MMU no tiene soporte para capturar hits).

- Requiere apoyo de hardware.
- Libre de la anomalía de Belady.
- Se puede implementar utilizando un time para cada página que se actualiza siempre que se accede a la misma, y se busca la de menor time. Esto requiere un poco de apoyo de hardware y luego buscar en la tabla la página víctima, usando un recorrido secuencial (O(n)).
- Otra implementación es con una lista doblemente enlazada con acceso al primer y último elemento. Cuando se referencia una página se mueve al inicio y para elegir una víctima se utiliza la del final.
- Generalmente, LRU es mejor que FIFO, pero es inimplementable por el hardware disponible.

6.7.5. MFU/LFU

Los algoritmos de más frecuentemente utilizada y menos frecuentemente utilizada, se basan en mantener un contador midiendo la cantidad de referencias a cada página.

- MFU: si una página fue empleada muchas veces, probablemente vuelva a ser empleada muchas veces más.
- LFU: una página que ha sido empleada pocas veces es probablemente una página recién cargada, y va a ser empleada en el futuro cercano.
- Son igual de caros de implementar que LRU y tienen peor rendimiento, por lo que no tienen gran implicancia en la realidad.

6.7.6. Aproximaciones a LRU

Dada la complejidad que presenta la implementación de LRU en hardware, los siguientes sistemas buscan una aproximación a éste.

- **Bit de referencia**: es un bit adicional en cada entrada de la tabla de páginas. La MMU prende el bit cuando usa la página y cada cierto tiempo, el SO puede apagar todos. Gracias a ello, se pueden distinguir las páginas que se usaron en el "último tiempo". Al momento de elegir, se define como víctima alguna que tenga el bit en 0 (si la hay). Si hay varios 0's o bien ocurre que son todos 1, se decide por FIFO, por lo cual también sufre de la anomalía de Belady.
- **Columna de referencia**: es una mejora del anterior, surge de la idea de guardar el historial antes de apagar todos los bits. Para ello, se agregan varios bits de referencia

a la tabla, y cuando se apagan todos, en realidad se desplaza la columna de bits a la derecha y se crea una nueva con 0's. Se puede implementar con corrimientos de bits. Cuando el sistema tenga que elegir a una nueva página víctima, lo hará de entre el conjunto que tenga un número más bajo.

- **Segunda oportunidad o reloj**: evita tener que poner todos los bits en 0 periódicamente. En cambio, se mantiene un apuntador a la próxima víctima y cuando el sistema requiera efectuar un reemplazo, éste verificará si el marco al que apunta tiene el bit de referencia encendido o apagado. En caso de estar apagado, el marco es seleccionado como víctima, pero en caso de estar encendido (indicando que fue utilizado recientemente), se le da una segunda oportunidad: el bit de referencia se apaga y el apuntador avanza en uno. En el peor caso, será también FIFO al igual que bit de referencia.

- **Segunda oportunidad mejorada**: además de usar el bit de referencia, utiliza el bit de dirty o escrito. En el mejor caso, ahorra la mitad del tiempo porque evita escribir en disco, ya que durante el tiempo que estuvo la página en memoria no fue modificada. Justamente por ello, siempre conviene elegir páginas que tengan el bit dirty apagado. Este bit lo apaga el SO cuando carga la página, y el hardware lo prende cuando se realiza una escritura. Las posibles combinaciones de bits son:

  - `(0,0)` No ha sido utilizado ni modificado recientemente. Candidato ideal para su reemplazo.
  - `(0,1)` No ha sido utilizada recientemente, pero está modificada. No es tan buena opción, porque es necesario escribir la página a disco antes de reemplazarla, pero puede ser elegida.
  - `(1,0)` El marco está limpio, pero fue empleado recientemente, por lo que probablemente se vuelva a requerir pronto.
  - `(1,1)` Empleada recientemente y sucia. Sería necesario escribir la página a disco antes de reemplazar, y probablemente vuelva a ser requerida pronto. Debe evitarse reemplazarla.

  También podría utilizarse el bit de RO, al revés que el de dirty (si está en 0, fue escrita).

En conclusión, la idea principal es tratar de buscar páginas que no fueron ni referenciadas (utilizadas hace poco) ni modificadas (escritas). Esas serán las mejores candidatas como víctimas.

6.7.7. Algoritmos con manejo de buffers

Son mecanismos que se emplean cada vez con mayor frecuencia, en los cuales el SO no espera el momento en que tenga que reemplazar un marco de memoria, si no que se encarga de siempre tener un poco de memoria disponible. Para ello, busca las páginas que tienen encendido el bit de dirty y va actualizando el disco y marcándolas como limpias.

6.8. Asignación de marcos

Un nuevo tema a abordar es cómo se asignan los marcos existentes a los procesos del sistema. Lo primero a tener en cuenta es que, si un proceso tiene asignados muy pocos marcos, su rendimiento indudablemente se verá afectado. Por ello, se garantiza una asignación mínima de marcos a cada proceso, reduciendo la frecuencia de fallos y mejorando la fluidez y eficiencia de la ejecución de los procesos.

En realidad ese es solo un comienzo, pues se debería buscar el nivel deseable de marcos para cada proceso, y no únicamente otorgarles el mínimo. Como bien sabemos, el rendimiento de un proceso será mejor mientras menos fallos de página cause. Sin embargo, tampoco tendría sentido otorgarles todos los marcos que necesita pues disminuiría la multiprogramación y por tanto el uso efectivo del CPU. Alternativas:

- **Asignación igualitaria:** dividir en partes iguales la memoria física entre todos los procesos en ejecución. Es deficiente porque ignora los distintos posibles tamaños de los procesos.
- **Asignación proporcional:** dar a cada proceso una porción del espacio de memoria física proporcional a su uso de memoria virtual. Este esquema en su forma más simple ignora las prioridades de los procesos. Además, cada vez que se inicia o finaliza un nuevo proceso, deben recalcularse todas las proporciones.

6.8.1. Ámbitos del algoritmo de reemplazo de páginas

- **Reemplazo local**: cuando se presente un fallo de página, las posibles víctimas serán aquellas pertenecientes al mismo proceso que el que causó el fallo. Por esto el espacio en memoria del proceso se mantiene estable.
- **Reemplazo global**: Se determina el espacio asignado para cada proceso cuando se inicializa, y éste influye en los algoritmos de reemplazo (es más probable que sea elegida como víctima una página que pertenezca a un proceso que excede de su asignación inicial). Los algoritmos que vimos anteriormente son de reemplazo global.
- **Reemplazo global con prioridad**: un proceso puede robarle espacio a otro solo si tiene mayor prioridad, y no necesariamente después le restituye.

Los sistemas con reemplazo global pueden llevar a rendimiento inconsistente: la misma sección de código puede tener tiempos de ejecución muy distintos si porciones importantes de su memoria fueron paginadas a disco.

6.9. Hiperpaginación, Sobrepraginación o Thrashing

Se refiere a una alta tasa de actividad de paginación, debida a que un proceso invierte más tiempo implementando los mecanismos de paginación que en la propia ejecución del proceso.

Es un problema de rendimiento en sistemas de memoria virtual, y lleva a que el SO pase la mayor parte del tiempo realizando swap al disco en lugar de ejecutar las tareas de los procesos. El thrashing es relativamente fácil de detectar, la CPU está libre y hay mucho intercambio (lecturas/escrituras al disco).

Esto ocurre porque dada una memoria física limitada, como podemos enviar a swap las páginas que no tengan un marco disponible, se puede terminar sobrecomprometiendo la memoria. Una salida sería reducir el nivel de multiprogramación: el sistema puede seleccionar uno (o más) de los procesos y suspenderlos por completo hasta que el sistema vuelva a un estado normal.

Ejemplo de hiperpaginación: Supongamos una memoria con capacidad para 4 páginas, y varios procesos ejecutando en este momento. Probablemente, si un proceso P1 está usando 2 paginas y otro proceso P2 usando las 2 restantes, y en ambos ocurren fallos, los procesos quedan en `sleep`. En algún momento, P1 se despierta, y le roba las páginas al segundo proceso. Ahora, cuando se despierte P2 le saca las páginas a P1, y así sucesivamente. En este caso, vemos que tenemos varios procesos en memoria, y mientras algunos duermen esperando conseguir sus páginas de swap, los otros les roban las páginas. Una peor situación se daría si en algún momento quedan todos los procesos durmiendo, y el SO carga otro proceso porque ve que la CPU está libre. Este es un claro ejemplo del caso en el que se tiene la memoria llena y los pedidos al swap son demasiados y conflictivos entre sí, teniendo como consecuencia que los procesos no puedan avanzar.

7. Memoria virtual en Linux

7.1. Organización de la memoria física

Tenemos distintas zonas de memoria. Aquellas que se asignarán para memoria interna del núcleo, o bien las que se guardarán para ser asignadas dinámicamente o las reservadas para el hardware. En los sistemas x-86 de Linux, se disponen los siguientes bloques:

1. Bloque [0 .. 0x0fff] (primeros 4 kb, 1 página): reservado para el hardware
2. Bloque [0x1000 .. 0x9ffff]: disponible para memoria dinámica.
3. Bloque [0xa0000 .. 0xfffff]: reservado para el hardware (dispositivos mapeados en memoria y rutinas de bios), este el concoido “agujero” desde los 640 Kb hasta 1Mb.
4. Bloque [0x100000 .. 0x100000+sizeof(kernel)]: núcleo.

5. Bloque [0x100000+sizeof(kernel) .. final]: disponible para memoria dinámica.

Cuando arranca la BIOS, busca un SO y lo carga en memoria disponible en la RAM. Generalmente, el núcleo no se carga desde las direcciones más bajas porque en el primer bloque (0x0 a 0x1000) no entra, y se prefiere utilizar memoria contigua.

Otras características de Linux en cuanto a su gestión de memoria:

- **Memoria compartida:** entre procesos (e hilos). Una página que está en memoria puede estar siendo referenciada desde una, dos o más tablas de paginación.
- **Copy on write:** utilizada por ejemplo en el Fork. Cuando un proceso genera un write en una pagina recién es cuando se realiza la copia.
- **Demand Loading:** las paginas de un programa binario se crean a medida que se necesitan.
- **Memory mapped files:** a través de determinadas llamadas a sistema, se puede pedir que un archivo este mapeado a memoria. Es decir, se puede acceder a un archivo como si fuera datos en memoria. Esto requiere un trabajo del kernel. El SO tiene que darse cuenta que un llamado a Read es a un MMF y tiene que leer de una dirección de memoria. Un caso particular de esto son las librerías compartidas.
- **Readahead:** se utiliza lectura por adelantado. Es probable que cuando se lea de un archivo, se vuelva a leer de un bloque contiguo dentro de un periodo corto de tiempo. Por eso, es preferible que el SO ya tenga algunos bloques de más cargados en memoria (ya leídos) y mejorar así la latencia. No se tienen garantías de que el programa vaya a usarlo particularmente, pero en caso de que finalmente lo haga, es una ventaja. Es por esto que se utilizan cachés de discos grandes.
- **Soporte para NUMA** (non uniform memory access): Se asigna débilmente una porción de la RAM a cada procesador, como si tuviera afinidad a esas direcciones. No es estricta esta asignación, pero permite mayor velocidad al acceder a direcciones de la memoria que son afines a ese procesador. Esto genera una cierta complejidad.
- **Reverse Mapping:** al implementar el algoritmo de swapping, es necesario almacenar más información sobre los marcos físicos. Para ello se utiliza el mapeo inverso.
- **Reservas en memoria:** Hay casos en los que se necesita memoria para una interrupción que es urgente. En estas ocasiones, si se necesita un marco vacío en memoria y no hay, el hacer swap hace muy lenta la interrupción porque depende de las velocidades del disco. Para evitar esto, se utiliza una reserva de memoria (`pool`). Básicamente, se hace un pedido especial de RAM, llamado *pedido de ubicación atómica* que resuelve la demanda en el momento y evita la demora.

- **Caches en RAM:** además de las conocidas caches de hardware, también puede destinarse parte de la RAM a caché. Esta distinción es indistinta, y generalmente mientras menos memoria es utilizada por procesos, más de ella se reservará para caches del sistema, mientras que si hay muchos procesos, buena parte de la RAM se utilizará para guardar información de ellos y poca para los caches.

7.2. Ubicación de memoria y gestión de memoria libre

Si bien Linux utiliza paginación en los espacios de direcciones de los procesos, el kernel no usa memoria virtual. Primero, porque necesita escribir en la memoria física en bloques contiguos, pues el DMA (acceso directo a memoria) no sabe de la existencia de la MMU, por lo tanto necesita memoria contigua. Por otro lado, sabemos que si las tablas de paginación se modifican frecuentemente, la TLB se invalida frecuentemente. En el caso del kernel, que ocurra esto penalizaría demasiado el rendimiento. Además, aunque se intentara aumentar los tamaños de las páginas para solucionar esto, se debería hacer lo mismo para todos los procesos y terminaría generando mucha fragmentación.

Por eso, para poder manejar la memoria y gestionar el espacio libre, se usan dos estrategias que se presentan a continuación.

7.2.1. Buddy Sistem y Slab Allocator

Existe un sistema muy popularizado, no propio de Linux si no anterior, que se denomina sistema de colegas. Este método es implementado para hacer el seguimiento de la memoria libre. En él, los huecos que quedan en memoria son exclusivamente de tamaño potencias de dos. Luego, si dos bloques son vecinos en memoria y tienen el mismo tamaño, pueden fundirse entre sí y generar huecos más grandes y con más probabilidad de ser utilizados. De esta manera, se soluciona el problema de la fragmentación externa. La estructura de datos utilizada para implementar este sistema es un arreglo de listas. Las listas contienen la memoria libre y hay una para cada tamaño de bloque.

Por otro lado, dado que el buddy system permite administrar huecos de por lo menos una página, no puede utilizarse para reservar memoria en porciones más chicas. Por esto existe el Slab Allocator, el cual se utiliza para la asignación eficiente de objetos pequeños y de tamaño fijo que son comúnmente utilizados por el núcleo.

7.3. Reclamo de páginas - Out of Memory

Existe un algoritmo de reclamos de marcos del núcleo, o PFRA (*page frame reclaiming algorithm*), que se encarga de rellenar la lista de bloques libres, robando marcos de procesos y de cachés del núcleo. Este algoritmo entra en juego cuando se está acabando la memoria del sistema. Es decir, aunque tiene sentido pensar que cuando no hay más memoria se reclaman

páginas, en realidad, para el funcionamiento del PFRA se necesitan algunas páginas en memoria, por lo que se debe hacer anteriormente (cuando los sectores libres comienzan a escasear). Esto se lleva a cabo conservando un pool de marcos libres mínimo.

El algoritmo maneja los marcos de diferente manera según su contenido. Existen 4 distinciones entre marcos:

1. **No reclamables:** Páginas que ya están libres o bien están reservadas, páginas correspondientes a la memoria dinámica del núcleo o al stack del núcleo, o páginas bloqueadas. Un ejemplo de páginas bloqueadas temporalmente son aquellas que están involucradas en concurrencias. Además, existen páginas bloqueadas en memoria, que se logran mediante llamadas especificas para asegurarse que no vayan a swap (como ejemplo se utilizan en bases de datos).

2. **Swapeables:** Páginas anónimas (que forman parte del espacio de MV de un proceso y no contienen datos provenientes de archivos) Por ejemplo el .text, .data, stack, etc.

3. **Sincronizables:** Páginas que contienen archivos mapeados en memoria, cachés de disco como el de i-nodos.

4. **Descartables:** Páginas no usadas del caché de memoria o del caché de direntries.

Mientras que a los marcos **no reclamables** debe ignorarlos y no elegirlos, los **descartables** puede seleccionarlos perfectamente. Por otro lado, los **swapeables** puede utilizarlos mientras guarde el contenido en swap y los **sincronizables** puede utilizarlos mientras sincronice su imagen con el disco, en caso de ser necesario.

7.3.1. Reglas del PFRA

En realidad, el algoritmo no sigue ninguna política en particular (como LRU, FIFO, etc), pero sí tiene en cuenta algunos criterios.

- Liberar primero las páginas que provoquen menos daño, por ejemplo las de cachés.
- Considerar reclamables todas las páginas de un proceso, salvo las bloqueadas en memoria.
- Antes de liberar una página compartida, se deben ajustar las entradas en las tablas de paginación de los procesos involucrados (mediante mapeo inverso).
- Reclamar páginas no utilizadas recientemente, utilizando el bit Accessed.
- Ahorrar la escritura en el disco, utilizando el bit de Dirty.

7.3.2. OOM Killer

Aunque estén implementados los mecanismos nombrados para realizar reemplazos de páginas y manejar la memoria de forma eficiente, a veces esto no es suficiente. El reclamo de páginas se realiza en forma periódica a través de un proceso *kswapd*, y también se recupera memoria de los buffers del slab allocator. Sin embargo, hay casos en los que igualmente la memoria libre es extremadamente baja. Si esto ocurre, se comienza a achicar los tamaño de los cachés.

Suponiendo ahora que la situación sea peor y además de no haber memoria, ni siquiera haya suficiente swap disponible para continuar las operaciones normales, el núcleo puede incluso eliminar un proceso sin ni siquiera enviarlo a swap. Para realizarlo, le encarga la decisión al Out of Memory Killer.

Básicamente, el OOMK es un mecanismo del núcleo de Linux diseñado para manejar de manera agresiva situaciones en las que el sistema se queda sin memoria RAM disponible. Aunque parece simple, la decisión de eliminar un proceso es muy complicada, porque no es "obvio"qué proceso matar. Se debe tener en cuenta que el `kill` se haga a un proceso dentro de todo nuevo, que no haya hecho mucho trabajo, que sea de un tamaño considerable, que no este involucrado en secciones críticas, etc.

7.4. Swap en Linux

Por último, se describirán tres características a tener en cuenta sobre el swap.

- **Caché de Swap:** Utilizado para mejorar el rendimiento, guarda páginas que se enviaron al espacio de intercambio, evitando muchas veces lecturas innecesarias al disco.
- **Multiple Swap-In:** El núcleo debe evitar que una página compartida que está en el disco sea cargada dos veces en memoria. Esto podría ocurrir si dos procesos que la comparten desean acceder a ella (piden un Swap In) y el SO no nota que uno de ellos hizo el pedido antes que el otro.
- **Swap-In y Swap-Out Concurrentes:** Se deben sincronizar adecuadamente las operaciones de swap para evitar conflictos. Si se envía una página a swap, y antes de que se escriba en disco el proceso intenta utilizar esa página, puede detectar que no está en memoria y traer del disco una versión anterior.

8. Sistemas de archivos

8.1. Conceptos básicos

Archivo: Es un tipo de datos abstracto, al cual solo pueden acceder los procesos por medio de la interfaz que brinda el SO. Un archivo es la unidad mínima lógica de almacenamiento

persistente (en el cuál nos enfocaremos). En sí, es solamente una secuencia ordenada de bytes.

Descriptor de archivo: Es un identificador; una vez que un archivo fue abierto, las operaciones que se realizan dentro de éste no son empleando su nombre, sino su descriptor de archivo.

Directorio: Dentro de cada volumen o medio de almacenamiento, los archivos disponibles conforman un directorio. El directorio es la estructura que relaciona a los archivos como son presentados al usuario –identificados por una ruta y un nombre– con las estructuras que los describen ante el SO. Aunque en realidad un directorio en Unix es simplemente un archivo más, con algunas características especiales.

Tabla de archivos abiertos: Tanto el SO como cada uno de los procesos, mantiene una tabla de este tipo, para guardar información por medio de un descriptor de archivo. Sin embargo, mientras que el SO debe llevar un conteo de usuarios del archivo, modos de acceso, ubicación en disco, e información de bloqueo, el proceso simplemente guarda el descriptor y los permisos.

8.1.1. Acceso concurrente

Ahora bien, en sistemas multitarea, ¿qué pasa cuando dos procesos quieren acceder a un mismo archivo? Se tienen dos clasificaciones sobre el acceso (podrían darse las 4 combinaciones de ellas, pero no todos los SO las implementan):

- **Acceso compartido vs exclusivo:** Por un lado, un lock compartido es un bloqueo que permite a varios procesos obtenerlo al mismo tiempo, garantizando que el archivo no sufrirá modificaciones. En cambio, el lock exclusivo puede ser adquirido por un sólo proceso, e indica que realizará operaciones que modifiquen los datos.
- **Acceso obligatorio vs consultivo:** En el bloqueo obligatorio el SO se encarga de imponer restricciones para la concurrencia, mientras que en el consultivo depende del programador la cooperación entre procesos.

Como regla general, en los sistemas `Windows` se maneja un esquema de `bloqueo obligatorio`, y en sistemas `Unix` es de `bloqueo consultivo`. Esto explica el que en Windows sea tan común que el sistema mismo rechace hacer determinada operación porque el archivo está abierto por otro programa (bloqueo obligatorio compartido), mientras que en Unix esta responsabilidad recae en cada uno de los programas de aplicación (bloqueo consultivo compartido).

8.1.2. Tipos de archivos

Naturalmente, hay archivos de diferentes tipos (de texto, binario, de audio, etc), y para el SO es fundamental poder reconocerlos. Esto puede lograrse por extensión, números mágicos o metadatos externos.

- **Extensión:** En el sistema cada nombre de archivo divide por un punto el nombre real con la extensión. Luego, el SO mantenía una lista de las extensiones conocidas y para ellas sabría como actuar. El problema de este esquema es que en determinados casos permite ocultar las extensiones permitiendo ataques de programas malignos que tienen extensiones falsas en el nombre del archivo. (utilizado por Windows).
- **Números mágicos:** Reconoce el contenido de un archivo basado en sus primeros bytes, denominados números mágicos. Un caso especial de estos es el llamado hashbang (#!). Esto indica a un sistema Unix que el archivo en cuestión debe tratarse como un ejecutable, y empleando como intérprete al comando indicado inmediatamente después del #!. (utilizado por Linux).
- **Metadatos externos:** Separa en dos divisiones la información del archivo: los datos que lo constituyen y la información acerca de él. La división de recursos incluye un campo llamado creador, que indica cuál programa fue el que generó al archivo. Si el usuario solicita ejecutar un archivo de datos, el SO lanzaría al programa creador, indicándole que abra al archivo en cuestión. (utilizado por Apple).

8.1.3. Métodos de acceso

La mayor parte de los SO maneja únicamente archivos sin estructura, y puede accederse a la información contenida en un archivo de varias formas distintas.

El **acceso secuencial** es el método de acceso más simple, en el cual la información se procesa por orden, un registro después de otro hasta llegar al final.

Sin embargo, si la única semántica por medio de la cual el SO permitiera trabajar con los archivos fuera la equivalente a una unidad de cinta, implementar el acceso a un punto determinado del archivo podría resultar demasiado costoso.

El hecho de que el SO no imponga estructura a un archivo no significa, claro está, que la aplicación que lo genera no lo haga. Para el **acceso aleatorio**, el archivo se considera como una secuencia numerada de registros, y no existe restricción alguna sobre el orden de acceso.

Por último, existen archivos débilmente estructurados que guardan registros de tamaño variable, por lo que no puede encontrarse la ubicación correcta por medio del acceso aleatorio. En el **acceso relativo a índice**, se guarda en una primera gestión una lista corta de identificadores, cada uno con el punto de inicio y término de los datos a los que apunta. Para leer un registro, se emplea acceso aleatorio sobre el índice, y el apuntador se avanza a la ubicación específica que se solicita.

8.1.4. Archivos especiales

Además de los archivos estándar propiamente dichos, existen los `objetos del sistema de archivos`. En ellos, se almacena la información acerca del sistema de archivos. En estos casos el SO oculta al archivo real y gestiona su información procesada. También se encuentran los archivos que permiten el `acceso directo a un dispositivo` externo. El contenido de este tipo de archivos son dos números que indican a qué dispositivo e instancia de él refiere. Además, los archivos pueden representar `comunicación entre procesos`, como pipes y sockets.

Al igual que en el manejo de memoria, en los archivos existen registros lógicos, y registros físicos (bloques). Generalmente, los tamaños de ellos no coinciden. Por ejemplo, el SO Unix define todos los archivos como simples flujos de bytes. Cada byte es direccionable de manera individual, a partir de su desplazamiento con respecto al principio del archivo. En este caso, el tamaño del registro lógico es un byte, y el sistema de archivos empaqueta automáticamente los bytes en bloques del tamaño necesario, como podrían ser 512 bytes por bloque.

8.2. Directorios

El directorio puede considerarse como una tabla de símbolos que traduce los nombres del archivo a sus correspondientes entradas de directorio.

8.2.1. Estructura

La estructura de directorio más simple es el **directorio plano** o de un único nivel. En este caso, todos los archivos están contenidos en un mismo directorio general. Sin embargo, en un sistema plano con varios usuarios, ellos no podrían tener archivos con nombres iguales, por lo cual tendrían que ser conscientes de lo que hace otro usuario. Por razones obvias, esto no es conveniente.

Se implementó entonces el **directorio en dos niveles**. En él se crea un directorio separado para cada usuario (cada uno tiene su propio `UFD - UserFileDir`) y hay un directorio maestro `MFD`. En esta estructura, cuando un usuario hace referencia a un archivo sólo se explora en su propio UFD.

El siguiente paso fue permitir una jerarquía ilimitada, definiendo los **directorios estructurados en árbol**. En este entonces cada directorio ya podía contener otros archivos o directorios anidados arbitrariamente. Con ello, nacen otros conceptos.

Como bien se sabe, cada usuario tiene un nombre, al igual que cada archivo. Ahora, un nombre de usuario y un nombre de archivo son suficientes para definir un nombre de ruta. Es decir, si el usuario A quiere acceder a su propio archivo "test"puede directamente referirse a ese nombre. En cambio, el usuario B deberá utilizar `/userA/test`, donde userA es el nombre de entrada del directorio de A. Entonces, se dice que las **rutas de búsqueda** son la secuencia

de directorios que se exploran cada vez que se proporciona un nombre de archivos. Además, los nombres de ruta pueden ser absolutos, si comienzan desde la raíz, o relativos, si comienzan desde el directorio actual.

Sin embargo, la solución de aislar a los usuarios, trae consigo la desventaja de no permitirles cooperar y acceder a archivos de otro. Por ello, surgió el directorio como **grafo dirigido**. En estos, cada directorio o archivo compartido existirá en el sistema de archivos en dos o más lugares simultáneamente. Es decir, el mismo archivo o subdirectorio puede estar en dos directorios diferentes. Un detalle, es que es común que la interfaz que presenta al usuario se restrinja en realidad a un grafo dirigido acíclico.

8.2.2. Hard links vs Soft links

La semántica de los sistemas Unix implementa directorios como grafos dirigidos por medio de dos mecanismos:

- **Enlace duro**: Lo que hacen es asociar dos o más ficheros compartiendo el mismo `i-nodo`. Esto hace que cada enlace sea una copia exacta del resto de archivos asociados, tanto de datos como de permisos, propietario, etc. Implica también que cuando se realicen cambios en uno de los enlaces o en el archivo también se realizarán en el resto de enlaces. Los enlaces duros no pueden hacerse contra directorios y tampoco fuera del propio sistema de archivos.
- **Enlace simbólico**: El fichero o directorio se encuentra en un único punto del disco y los enlaces son un puntero contra él. Cada enlace tiene su propio número de `i-nodo` lo que permite hacer enlaces entre distintos sistemas de archivos. Si borramos el archivo o directorio origen, el enlace permanece pero los datos desaparecen para siempre.

Ejemplo práctico:

```
ln original.txt hard_link.txt
ln -s original.txt hard_link.txt
stat
ls -li
```

Con el primer comando, se realiza un enlace duro al archivo original mientras que en el segundo comando se realiza un enlace simbólico. Utilizando stats o bien ls es posible ver que en el primer caso efectivamente comparten inodo y que los datos se sincronizan entre ambos. En cambio, con el segundo verificamos que el enlace simbólico tiene un inodo distinto, y contiene una referencia al archivo original.

En los SO (tanto Unix como Windows), todo directorio tiene dos entradas especiales: los directorios **.** y **..**, que aparecen tan pronto como el directorio es creado, y resultan fundamentales para mantener la navegabilidad del árbol. Mientras que **.** es una liga dura al mismo directorio, **..** es una liga al directorio padre (de nivel jerárquico inmediatamente superior).

Un detalle es que sólo puede haber una liga **..**, es por ello que un directorio enlazado desde dos lugares distintos sólo apunta hacia uno de ellos con su enlace. Sin embargo, Hay una excepción a esta regla: el directorio raíz. En este caso, tanto **.** como **..** apuntan al mismo directorio. Esta es la razón por la cual no se puede tomar rigurosamente un árbol de archivos como un grafo dirigido acíclico, tanto las entradas **.** como las entradas **..** crean ciclos.

8.3. Discos duros

Un *file system* sirve para **gestionar información persistente** que guarda en algún lugar físico. Debe permitir además consultarla, actualizarla y eliminarla. Todo FS contará con información que tiene que guardar (`data`) e información extra para la gestión (`metadata`). Esto es por ejemplo información sobre un archivo (el nombre, la fecha, el tamaño, dónde están sus bloques) al igual que información del sistema (qué bloques están libres, etc).

Ahora bien, ¿cómo se guardan esos datos? Se utilizan dispositivos que almacenan bloques, es decir, conjuntos de N bytes, generalmente de 1Kb o 0.5Kb. Observamos que esta es una primera diferencia con la RAM, la cual es accesible de a bytes. La persistencia que nombramos anteriormente es a nivel de estos sectores.

Se implementa una tecnología llamada LBA (*logical blocking address*), que le permite al disco almacenar el bloque en cualquier parte de él. Esta tecnología permite ganar rendimiento y aprovechar más el almacenamiento.

Lógicamente, el disco puede dividirse en **particiones**. Cada una de estas particiones puede tener un FS distinto. Por otro lado, se define un **volumen** como una colección de bloques (o partición) ya inicializada con un FS.

Por otro lado, físicamente el disco se particiona siguiendo una geometría de bloques. La misma define varias estructuras

- **Pistas**: los discos (o platos) de un disco duro tienen dos lados (caras) para registrar los datos. Cada superficie del disco tiene círculos concéntricos invisibles en ella, denominados pistas. Es decir, una pista es una circunferencia dentro de una cara; la pista cero (0) está en el borde exterior.
- **Sector**: es la unidad básica de almacenamiento de datos sobre los discos duros. Las pistas están divididas angularmente en sectores, y el número de sectores es variable.
- **Cilindros**: es un conjunto de varias pistas, son todas las circunferencias que están alineadas verticalmente (una de cada cara), las cuales están situadas unas encima de las otras. Lo que se logra con esto es que la cabeza no tiene que moverse para poder acceder a las diferentes pistas de un mismo cilindro.

Dentro del disco, la velocidad de rotación es rápida mientras que el tiempo que lleva cambiar de pista es mayor. Por eso, los sectores contiguos se leen mas rápido que los que están en cilindros distintos.

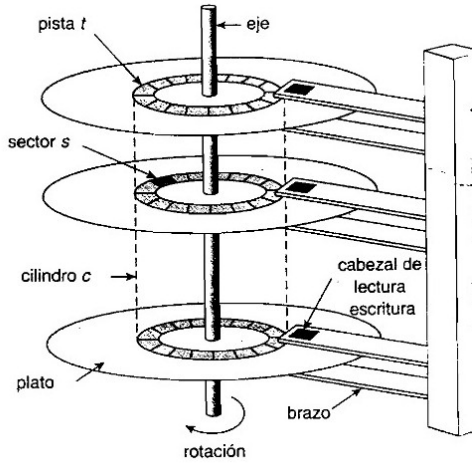

Figura 1: Geometría de bloque de un disco.

8.4. Algunos sistemas de archivos

8.4.1. TAR (Unix)

El sistema TAR consigue su nombre por el *tape archiving*. Es un sistema para guardar archivos en cintas, y es utilizado hoy en día para hacer back-ups cuando hay mucha información que guardar. En este sistema, se realiza el acceso de manera secuencial.

- Los metadatos (nombre, permisos, tamaño) se escriben en un sub-bloque llamado cabecera, antes de cada archivo.
- Es simple, pero ineficiente.
- Si quiero saber si un archivo existe antes de crear uno nuevo, tengo que recorrer si o si toda la cinta.
- Actualización compleja, es difícil cambiar el tamaño del archivo.

8.4.2. Sistema de commodore 64

Para intentar concentrar la metadata en un lugar de fácil acceso, se escribe al principio todo el conjunto de cabeceras. Por ello, se agrega en cada una dónde comienza el archivo. (La cabecera N apunta al archivo N).

- Sigue siendo simple, y ahora es más rápido saber si un archivo existe.
- El sector de cabeceras tiene un tamaño fijo. Una vez que ya esta creado el FS, no puedo agregar mas archivos que los que ya están en la cabecera.

- Actualización compleja, es difícil cambiar el tamaño del archivo.

8.4.3. QUESTAR/MFS

En los anteriores, si se quería aumentar el tamaño de un archivo no había lugar. Ahora, los archivos están fragmentados, y se puede aumentar su tamaño. Se denomina fragmento a un grupo de sectores contiguos del disco. Para cada fragmento, se guarda donde comienza y cuántos sectores tiene.

- Añade una forma de manejar el crecimiento de archivos.
- Nuevamente la cabecera tiene un tamaño fijo, por lo que la fragmentación es acotada.
- Se agrega al comienzo un mapa de bits para saber qué bloques están libres. (debido a la fragmentación ahora es necesaria esta estructura).

8.4.4. MSDOS FS

Se definió una estructura llamada **FAT (File Allocation Table)**. Es un arreglo de enteros, con una correspondencia uno a uno con cada cluster (conjunto de 4 sectores contiguos). El valor de cada entero dice si está libre, dañado o si es parte de un archivo.

Un **cluster** es una forma lógica de agrupar sectores, transparente al disco. Se usa ese tamaño de información para escribir las cosas en el disco, es decir, si se necesitan escribir 2 bytes, se solicitan igual 4 sectores. Claramente, esto genera mas fragmentación interna. Sin embargo, ayuda al rendimiento. Por un lado, fue necesario cuando comenzaron a usarse discos mas grandes, ya que permitía que la FAT (que tiene que estar en memoria) tenga menor tamaño (si los clusters son de 4 bloques, es 4 veces menor). Además, permite que haya menos fragmentación externa, ya que los bloques si o si son del tamaño de un cluster. Por último, asegura más espacio contiguo al archivo para leer o escribir.

- Buscar un cluster libre implica recorrer la FAT buscando un 0.
- Hay un sector de arranque, dos copias de la FAT por seguridad, luego el directorio raíz (simplemente un conjunto de dir-entries) y luego el resto de los datos.
- Es un buen sistema para discos chicos, ya si hay muchos clusters la FAT pasa a tener millones de entradas.
- Por fragmentación interna puede perderse hasta 4 bloques por archivo (1K en promedio).
- La sincronización con la FAT no es atómica. Puede ocurrir que se guarda la dir-entry pero no llegó a escribirse la FAT por algún motivo. En ese caso, se guardó un dato en un lugar que para el cluster seguirá estando disponible.

- Para que funcione correctamente la FAT debe estar cargada en memoria, lo cual puede ser muy costoso.
- En un comienzo, se asignaba un espacio fijo al directorio raíz, pero para FAT32, el directorio raíz ya no estaba ubicado en un espacio reservado, sino que era parte del espacio de datos, por lo cual resultó extensible en caso de requerirse.

¿Qué implica **crear un archivo** en este sistema? Cómo se mencionó, esto no es atómico, y en particular conlleva dos tareas:

1. Crear una dir-entry y buscar una entrada libre en la FAT.
2. Escribir la FAT con un valor.

Ahora bien, ¿cuál conviene realizar primero? La respuesta es que el orden deseado es 2 - 1. Suponiendo que se cree primero la dir-entry, pero ocurre algo en el medio y no puede marcarse en la FAT, va a aparecer como libre un espacio que está ocupado, y probablemente se use para otro archivo. Esto sería un grave error.

Por otro lado, si primero se marca la FAT con un valor, y antes de que se escriba la dir-entry ocurre un fallo, quedarán marcados como ocupados sectores que en realidad no lo están. Sin embargo, este es un problema menor, pues existen softwares y políticas que buscan estos espacios que no están siendo utilizados.

8.4.5. Unix FS

El disco comienza con un sector de arranque, luego un superbloque, una tabla de i-nodos (index-nodes), y el directorio raíz. El **superbloque** tiene la información de cómo está estructurado el resto del disco. Por otro lado, cada **i-nodo** tiene toda la información de su archivo menos el nombre del mismo, que está en el directorio. Es decir, la dir-entry está compuesta por el nombre del archivo y el i-nodo.

- Ahora las dir-entries son mucho mas sencillas, por lo tanto también es mas fácil buscar un archivo en un directorio.
- Algo muy práctico es que todos los i-nodos sean chicos y del mismo tamaño.
- El i-nodo es el sector lógico mínimo que toma el SO, y en él están todos los metadatos del archivo.
- En el mismo i-nodo está la información sobre los primeros 8 (o 12) sectores del archivo (nivel 1). Luego, hay punteros a tablas con los próximos sectores. Hay un puntero a una tabla con 128 sectores, luego un puntero a una tabla que contiene punteros a tablas de 128 sectores (128^2 sectores, nivel 2). Para los primeros discos esto era suficiente, sin embargo luego se agrego un tercer nivel de dirección.

- Cuando se tiene un archivo nuevo, se debe actualizar el mapa de bits, la dir-entry y el i-nodo, a comparación de MSDOS que solo necesita dos accesos (FAT y dir-entry).
- Un archivo puede tener varios nombres (links). Solo necesitan estar apuntando al mismo i-nodo. Por eso, borrar un archivo en este FS es borrar un link y decrementar el numero de links. Cuando este llega a 0 puedo eliminar el archivo.
- Una gran ventaja es que un i-nodo debe estar cargado en memoria solamente cuando el archivo correspondiente está abierto.

8.4.6. Mejorando el File System de Unix

Para mejorar el sistema en cuanto a su rapidez, se hizo crecer el sector lógico y se implementó el algoritmo del ascensor.

- **Sector lógico**: En la primera medida, se definió un sector lógico distinto del físico. Ahora, en vez de pedir de a 1 sector, el SO pedía 8 sectores físicos a la vez. Luego, cuando un programa pedía leer de memoria contigua a la que había solicitado anteriormente, el SO ya lo tenía en caché.
- **Algoritmo del ascensor**: También denominado SCAN, plantea una forma más eficiente de planificar las lecturas/escrituras al disco. El cabezal comienza en uno de los extremos del disco y se mueve hacia el otro extremo, dando servicio a las solicitudes a medida que pasa por cada cilindro, hasta llegar al otro extremo del disco. En ese otro extremo, se invierte la dirección de movimiento del cabezal y se continúa dando servicio a las solicitudes. El brazo del disco se comporta como un ascensor dentro de un edificio, primero dando servicio a todas las solicitudes de subida y luego invirtiendo su movimiento para dar servicio a las solicitudes de bajada.

Otra mejora fue distribuir el superbloque, en vez de guardarlo al principio del disco, se distribuyó en distintas partes. Luego, una operación tiene que ir al más cercano a ella. Es además más seguro porque si se cae uno de ellos, solo perjudica algunos sectores.

Por último, se modificó la estructura de los directorios, utilizando árboles o tablas hash para accesos con menor costo.

8.4.7. La organización del Unix FS

En el sistema de archivos Unix original, se dividen los discos físicos en discos lógicos llamados particiones, donde cada partición corresponde a un sistema de archivos independiente. Se diferencian entre ellos utilizando una combinación de números, esto es, a cada dispositivo de disco se le asigna su propio número de dispositivo mayor, y cada partición tiene un número de dispositivo menor.

Cada sistema de archivos, en este caso, contiene:

1. Un programa de arranque, que tiene que poder leer el file system de linux.
2. Un superbloque, que guarda todo el estado del file system.
3. La tabla de i-nodos (el mapeo uno a uno con los archivos).
4. El contenido real del archivo, en bloques de datos.

Como se mencionó anteriormente, el i-nodo contiene toda la información del archivo menos su nombre. Esto es así debido a que el nombre no se considera parte del archivo en sí, si no una forma de referenciar sus contenidos. Se podría decir que mientras el usuario piensa en los archivos por sus nombres, Unix los piensa en términos de i-nodos.

En el caso de los directorios, sus i-nodos se parecen al i-nodo de cualquier otro archivo. El contenido (bloques de datos) de un directorio incluye pares (nombre - i-nodo) de los archivos y subdirectorios dentro de él, mientras que el i-nodo del archivo en sí apuntará a los bloques directos que contienen los datos reales de él.

Como bien sabemos, el sistema de archivos Unix asigna bloques de datos (bloques que contienen el contenido de un archivo) uno a la vez desde un conjunto de bloques libres, en particular Unix utiliza bloques de 4K (al principio utilizaba de 512 bytes). Además, los bloques de un archivo están dispersos aleatoriamente dentro del disco físico. Por ello, los i-nodos incluyen punteros a los bloques de datos. Cada i-nodo contiene 15 punteros:

- Los primeros 12 punteros apuntan directamente a los bloques de datos.
- El 13º puntero apunta a un bloque indirecto, un bloque que contiene punteros a bloques de datos.
- El 14º puntero apunta a un bloque doblemente indirecto, un bloque que contiene 128 direcciones de bloques indirectos simples.
- El 15º puntero apunta a un bloque triplemente indirecto (que contiene punteros a bloques doblemente indirectos, etc).

Sin embargo, este sistema tenía varias **desventajas**:

- Los i-nodos de archivos en un directorio común no se mantienen juntos, lo que lleva a un bajo rendimiento al buscar en directorios.
- El sistema de archivos original solo utilizaba bloques de 512 bytes, un tamaño de transferencia ineficiente.
- Los bloques de datos de un archivo no se almacenan juntos, lo que lleva a un rendimiento deficiente al acceder a archivos secuencialmente.
- La lista de bloques libres se desorganiza rápidamente, aumentando la sobrecarga de encontrar bloques libres (búsqueda para cada nuevo bloque).

Es por ello que se planteó un sistema un poco mejorado.

8.4.8. Fast File System (Berkeley)

Se basó en los siguientes principios para mejorar el sistema:

- Duplica el superbloque (si hay un fallo se puede recuperar fácilmente).
- Los bloques son más grandes (lecturas contiguas).
- Se guarda el tamaño de los bloques en el superbloque.
- Intenta agrupar los datos en bloques de cilindros o **grupos de cilindros** (son los lugares donde la cabeza lectora puede leer casi sin moverse). Gracias a esto, ahora los datos están más cerca de su i-nodo. El mecanismo se basa en que recién cuando el primer grupo de cilindros está relativamente lleno, se continúa llenando el segundo. Igualmente, siempre se deja un poco de espacio en cada grupo para que los archivos puedan crecer.
- Para cada grupo de cilindros, se tiene una estructura que contiene una tabla de i-nodos, una tabla de bloques y fragmentos libres del grupo, y una copia del superbloque. Esto agrega la ventaja de que en caso de que se rompa un mapa de bits, se rompe solo ese grupo de cilindros, y no todo el disco.
- Se puede **compartir un mismo bloque de datos** entre distintos archivos, ya que permite guardar fragmentos (una cantidad dada de fragmentos forman un bloque). Básicamente, en los i-nodos hay una máscara que dice, del último bloque de datos asignado a este i-nodo, cuánto realmente se usó. Esto permite que un archivo pueda usar una parte del bloque y el resto dejarlo libre para que lo usen otros archivos. Cuando esto no existía, al usarse por ejemplo bloques de 4K, los archivos chicos desperdiciaban mucho lugar (fragmentación interna).

La principal ventaja de este sistema es la mejora que ofrece en el rendimiento, ya que todo está mucho más cerca.

Por otro lado, cuando se asigna espacio, el sistema de archivos rápido utiliza una **política global** para determinar dónde colocar nuevos directorios y archivos, y una **política local** para asignar bloques a niveles inferiores.

Globalmente:

- Colocar los i-nodos de los archivos en el mismo directorio en el mismo grupo de cilindros (lo que hace que programas como ls sean más rápidos).
- Colocar los nuevos directorios en un grupo de cilindros que tenga un número de i-nodos libres mayor que el promedio y el menor número de directorios ya presentes en él.
- Intentar colocar todos los bloques de datos de un archivo en el mismo grupo de cilindros.

- Mover a un nuevo grupo de cilindros cuando un archivo excede los 48 KB, y cada megabyte después de eso.

Localmente:

- Al agregar un bloque de datos a un archivo existente, elegir el siguiente bloque que esté más cerca rotacionalmente.
- Intentar asignar bloques dentro del mismo cilindro antes de mirar otros bloques del grupo.

8.5. Control de acceso

Según que modelo de sistemas de archivo se utiliza, hay distintos esquemas de control de acceso a los archivos.

8.5.1. Sistemas FAT

Por un lado, en los sistemas que utilizan la estructura FAT, cada entrada del directorio tiene el nombre, las fechas de creación y modificación, el número de cluster del disco donde comienza el archivo, y un byte con atributos. Los mismos son:

- `Oculto`: El archivo no se deberá mostrar al usuario en listados de directorio.
- `Sólo lectura`: El SO debe evitar toda modificación al archivo.
- `Sistema`: No se puede mover ni fragmentar al archivo.
- `Archivado`: Conforme se realizaba un respaldo, todos los archivos iban siendo marcados como archivados. Y cuando un archivo era modificado, el sistema operativo retiraba este atributo. De esta manera, el siguiente respaldo contendría únicamente los archivos que habían sido modificados desde el respaldo anterior.

8.5.2. Modelo tradicional Unix

A diferencia del sistema que utiliza la tabla FAT, el modelo de Unix sigue un esquema multiusuario. Por ello, en este SO cada usuario pertenece a uno o más grupos, gestionados por el administrador.

Cada archivo describe sus permisos con nueve bits, dispuestos en tres grupos de tres. Cada uno de los grupos se corresponde con `usuario`, `grupo` y `otros`, respectivamente. Dentro de ellos, los 3 bits representan permiso de `lectura`, `escritura` y `ejecución`. Para los directorios, se emplea el mismo sistema de permisos: escritura indica quién puede crear o eliminar archivos del directorio, lectura indica qué usuarios pueden averiguar la lista de archivos del directorio, y el de ejecución indica qué usuarios pueden entrar al mismo.

8.5.3. Listas de control de acceso

En el modelo anterior, hay dos desventajas importantes: por un lado, para crear grupos se necesita al administrador. Por otro lado, el hecho de que un archivo no puede pertenecer a más de un grupo es un gran inconveniente. Encontrar la representación justa para indicar un conjunto específico de permisos puede ser complicado.

Por ello, se agregaron las listas de control de acceso (ACL). Con ellas, cada uno de los archivos tiene asociada una lista donde se indican los permisos particulares que tendrá sobre éste cada usuario o grupo particular. Todos los permisos del archivo se almacenan en su i-nodo, representados como una estructura adicional adjunta a los datos del archivo.

8.6. Sistemas de archivos remotos

Una pregunta interesante que surgió en la década de los 80 fue cómo compartir archivos entre computadoras independientes. Para poder permitir esto, se desarrollaron distintos sistemas de comunicación en red. La idea general es, si bien el SO no tiene acceso directo a los archivos y directorios que le solicitará el usuario, por medio de los módulos de red sabe cómo obtenerlos y presentarlos como si fueran locales.

8.6.1. NFS vs CIFS

Aunque tanto **Network File System** (NFS) como **Common Internet File System** (CIFS) son protocolos de acceso a archivos, son muy diferentes. Primero, CIFS está dirigido al sistema operativo Windows, mientras que NFS está dirigido a Unix. Si bien es posible utilizar CIFS con sistemas operativos basados en Linux (a través de Samba) y NFS con sistemas operativos Windows (a través de paquetes de terceros), no son compatibles de forma nativa.

NFS: El sistema de archivos que ofrece sobre la red NFS cumple con la semántica Unix completa, para emplear un sistema remoto, basta montarlo y usarlo como si fuera local. El manejo de permisos, usuarios, e incluso las ligas duras y simbólicas se manejan exactamente como se haría localmente. Dado que se espera que la información de usuarios y permisos sea consistente en todos los clientes, se ofrecía también un esquema llamado NIS, Network Information System, para compartir la información de autenticación y listas de usuarios. Por último, para garantizar que las escrituras a archivos se llevaran a cabo cuando eran solicitadas, todas las escrituras en un principio sobre NFS eran manejadas de forma síncrona.

CIFS: Fue pensado originalmente para una red pequeña, con hasta un par de decenas de equipos. La mayor parte de los paquetes eran enviados en modo de difusión (broadcast), por lo que era fácil llegar a la saturación, y no había un esquema centralizado de resolución de nombres, por lo que era frecuente no encontrar a determinado equipo. La única medida

de seguridad que implementaba era el manejo de hasta dos contraseñas por directorio compartido. Con una el usuario obtenía acceso de sólo lectura, y con la otra de lectura y escritura.

Además, mientras que NFS se limita a compartir archivos y directorios, por el contrario, CIFS también puede ofrecer comunicaciones de recursos compartidos con otros dispositivos de red, como impresoras. Notamos algunas diferencias clave entre ellos:

- **Bloqueo de archivos**: NFS no hace un seguimiento de los archivos abiertos en el servidor, por lo que el usuario debe gestionar el bloqueo de archivos. En cambio, CIFS evita por sí mismo los conflictos cuando varios usuarios intentan acceder al mismo archivo.
- **Rendimiento**: NFS utiliza un protocolo ligero, con menos sobrecarga y mayor velocidad de transferencia y acceso a los archivos. En cambio, CIFS tiene un protocolo de mayor sobrecarga.
- **Autenticación**: NFS controla el acceso a los archivos compartidos únicamente en función de la dirección IP del cliente, teniendo mecanismos de seguridad limitados. En cambio, CIFS admite la autenticación basada en el usuario junto con mejores mecanismos de seguridad.

8.6.2. Andrew File System

Plantea presentar un verdadero sistema de archivos distribuido, en el cual los recursos compartidos no tengan que estar en un servidor en particular, sino que un conjunto de equipos se repartan la carga.

- El uso típico de AFS se planteaba para organizaciones grandes, del orden de decenas de miles de estaciones.
- Busca una fácil escalabilidad, la capacidad de agregar tanto espacio de almacenamiento como equipos con papeles de servidor.
- Debe operar tan confiablemente como sea posible, incluso sin la certeza de que la red opera correctamente.

En AFS, los cambios a un archivo abierto por un usuario no son visibles a los demás de inmediato. Sólo una vez que se cierra un archivo, los cambios hechos a éste son puestos a disposición de las sesiones abiertas actuales, y sólo son enviados como versión actual a las sesiones abiertas posteriormente. Bajo el esquema aquí descrito, las lecturas y escrituras resultan baratas, porque se realizan exclusivamente sobre el caché local, pero abrir y cerrar un archivo puede ser muy caro, porque debe transferirse el archivo completo.

9. Sistemas Distribuidos

Un sistema distribuido, o SD, se forma cuando hay varias CPUS interconectadas en una red (generalmente de alta velocidad, pero no es un requerimiento necesario). Ahora bien, una pregunta que surge es, ¿qué diferencia un SD de una CPU con muchos procesos ejecutando simultáneamente? La idea general es que cuando se tiene una única CPU, todos los procesos comparten memoria. En cambio, en SD no hay memoria compartida, y tampoco hay una seguridad de que los mensajes lleguen siempre ni en orden.

Entonces, ¿por qué se usan los SD?

- **Rendimiento**: esta es la razón más importante. Por más rápido que sea un procesador, no será más eficiente que muchas computadoras (lentas) juntas.
- **Distribución**: muchas veces, es necesario por temas de cercanía, tener algún tipo de distribución. Esto es porque no solo el sistema de cómputo está distribuido si no también los datos. Ejemplo: google, un servidor en cada país.
- **Seguridad**: al permitir tener muchas copias de los datos y el servicio, se logra una alta disponibilidad, lo cual brinda más confiabilidad.
- **Crecimiento**: en SD se permite un crecimiento progresivo, pues se pueden agregar unidades de cómputo al sistema según lo vaya necesitando. Además, por el lado contrario, también pueden sacarse computadoras si no se están utilizando.

Sin embargo, el optar por un SD añade una gran complejidad, por los siguientes motivos

- Es importante brindar una buena planificación de la división de tareas y la programación paralela para que el sistema funcione correctamente.
- Los mensajes utilizados para la comunicación dependen de una red, lo cual no garantiza confiabilidad en cuanto a tiempos, orden, etc.
- Es de fundamental importancia tener cuidado sobre quien tiene acceso a los datos, pues los mismos están disponibles en la red.

En cuanto a nociones de transparencia, ninguna es completamente obligatoria. El usuario no necesita saber cuánta concurrencia hay en un momento determinado, ni tampoco cómo se esta ejecutando el programa, qué tan paralelo, dónde están los recursos, o cuántas copias hay de ellos. Esto mismo es algo muy difícil de lograr.

En estos sistemas, se utilizan algoritmos distribuidos, para dar acceso exclusivo a un recurso si es compartido.

9.1. Algoritmos distribuidos

9.1.1. Algoritmo centralizado

Hay un nodo coordinador, y cualquier otro nodo que necesite acceder a un recurso compartido le enviará un mensaje al coordinador, para que le permita el acceso si no hay otro proceso en la sección crítica. Además, cuando el proceso termina de utilizar el lock enviará otro mensaje al coordinador para notificarle. Si un proceso requiere un lock pero el coordinador se lo dio a otro, puede o bien no responderle hasta que esté disponible, o responderle un acceso denegado. En caso de que se tome la primer medida, puede ocurrir que el coordinador se desconecte o apague y el proceso quedará en una espera infinita. Si se utiliza la segunda medida, se genera *busy waiting* porque el proceso seguirá intentando acceder al recurso. Por lo tanto, hay un mayor tráfico en la web. Por último, es necesario aclarar que el algoritmo tiene problemas de cuello de botella con el nodo coordinador, y si falla el mismo, se rompe todo el sistema.

9.1.2. Algoritmo de Lamport, Ricart y Agrawala

En este algoritmo, se asume que la mensajería es confiable. Además, requiere que todos los eventos en el sistema tengan un orden total, basado en un *timestamp*. Cuando un proceso necesita un acceso a una región crítica, envía un mensaje a todos (broadcast) incluyendo qué región crítica, la hora y su identificador de proceso. Luego, esperará que todos respondan OK. Si un proceso recibe el mensaje y no quiere entrar, responde OK. Si está en este momento en la región crítica, espera a salir para responder OK. Por último, si también tiene intención de entrar, compara las horas (con su solicitud) y responde OK en caso de que el timestamp sea menor al suyo.

Es decir, cuando un proceso sale de la RC, envía OK a todos los procesos esperando por ella.

Este algoritmo surge para mejorar el problema del único punto de falla en el centralizado, pero termina teniendo n (cantidad de nodos) puntos de falla. Aunque puede mejorarse esperando únicamente una mayoría de mensajes y no todos, pero de cualquier manera es más lento, más complicado, más caro y menos robusto que el anterior.

9.1.3. Algoritmo basado en anillo

Se ordenan lógicamente los nodos/procesos en un anillo, y un token (paquete especial) circulará por el mismo. Luego, sólo el proceso que lo tenga podrá acceder, y lo soltará cuando deje de estar en la región crítica. Este algoritmo sí es conveniente utilizar en vez del centralizado.

9.2. Sistema Operativo Distribuido

9.3. Detección de deadlocks

En el algoritmo centralizado, un intento para detectar deadlocks es con el grafo pedido-tomado. En este caso, un proceso coordinador lleva el estado del grafo y cada proceso le avisa cuando quiere agregar o eliminar un arco hacia un recurso. Si el coordinador detecta que se originó un ciclo, entonces avisa el proceso para que aborte. Sin embargo, hay escenarios en los que se concluye falsamente que hay un deadlock. En la mayoría puede solucionarse utilizando timestamps, pero lo hace más costoso.

Por otro lado, en el algoritmo distribuido, existe el algoritmo Candy-Misar-Haas. La idea básica es enviar un mensaje que intenta recorrer dinámicamente y en forma distribuida el grafo de pedidos/tomados. Si un proceso debe esperar por recursos, se lo indica a los procesos que los poseen. Si hay un ciclo, eventualmente algun proceso enviará un mensaje al primer proceso nombrado, y al recibirlo, éste podrá detectar un deadlock.

10. Bibliografía

1. Fundamentos de sistemas operativos - Gunnar Wolf, Esteban Ruiz, Federico Bergero, Erwin Meza - año 2015

2. Apuntes de la cátedra Sistemas Operativos 2.

3. https://web.cs.wpi.edu/ rek/DCS/D04/UnixFileSystems.html

Hecho por Zoe durante el cursado de la materia en 2024